PROJET COMPLEXE 2026 REVIVAL

ASC + TAURI SOLIDJS SECOND BRAIN

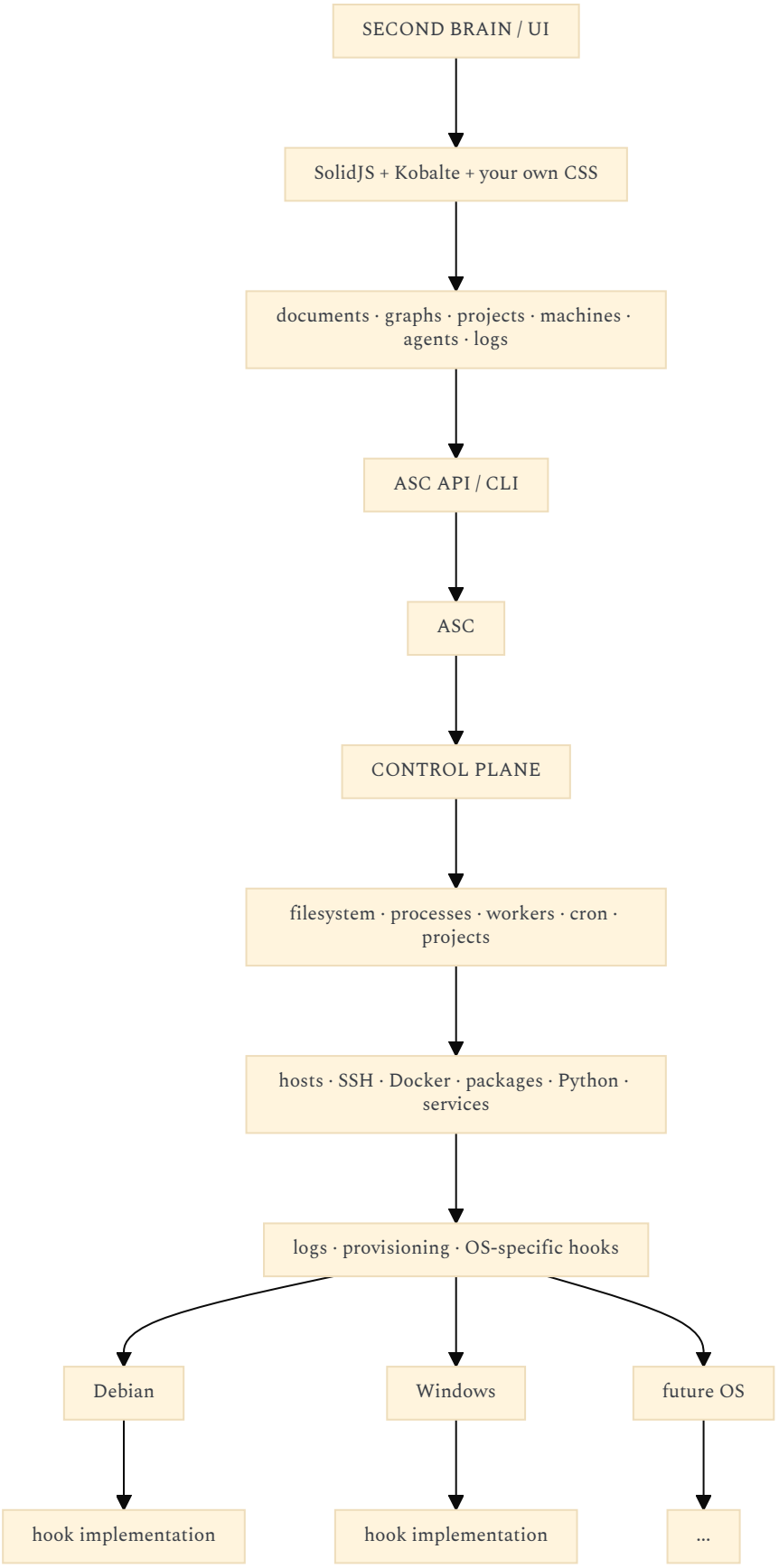
https://github.com/Paulmicha/asc
https://github.com/Paulmicha/projet-complexe
https://chatgpt.com/s/t_6a7cce6079d88191804cecdac5cc21e1

FIRST SHOT

You are not really building a Tauri application that happens to have a backend. You are building a **visual control plane over ASC**, where Tauri/Solid is deliberately kept thin and ASC remains the authority for everything that touches the machine.
I think this is the stronger architecture.
The key rule should be:

> **Tauri never directly operates the host. It asks ASC to operate the host.**

That gives you a very clean separation:



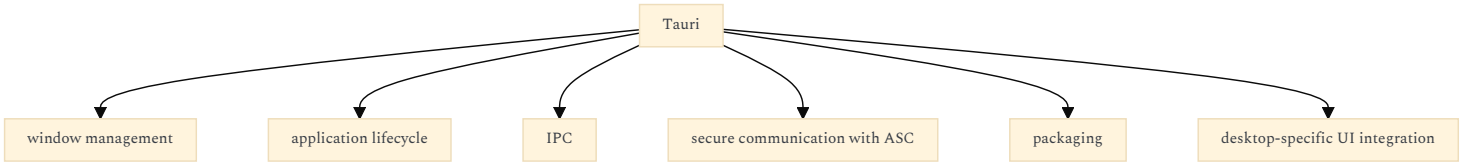
This also makes your original idea of ASC much more coherent: **ASC becomes the machine abstraction, while the second-brain application becomes the machine's visual/intellectual interface.**

1. I WOULD CHANGE ONE THING FROM MY PREVIOUS ANSWER

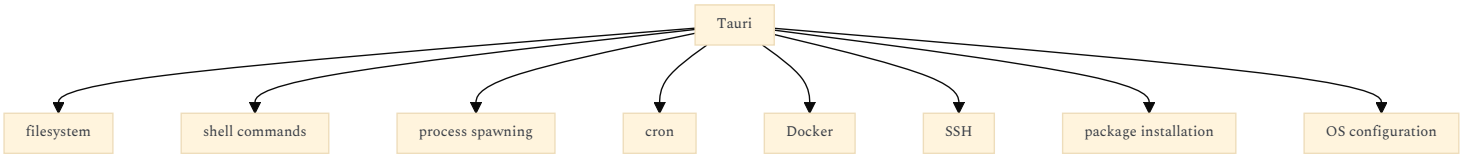
I previously proposed a Rust layer containing things such as:

- filesystem
- process management
- background workers
- OS integration
- ASC integration

I would now remove almost all of that.
Your Tauri layer should be much thinner:



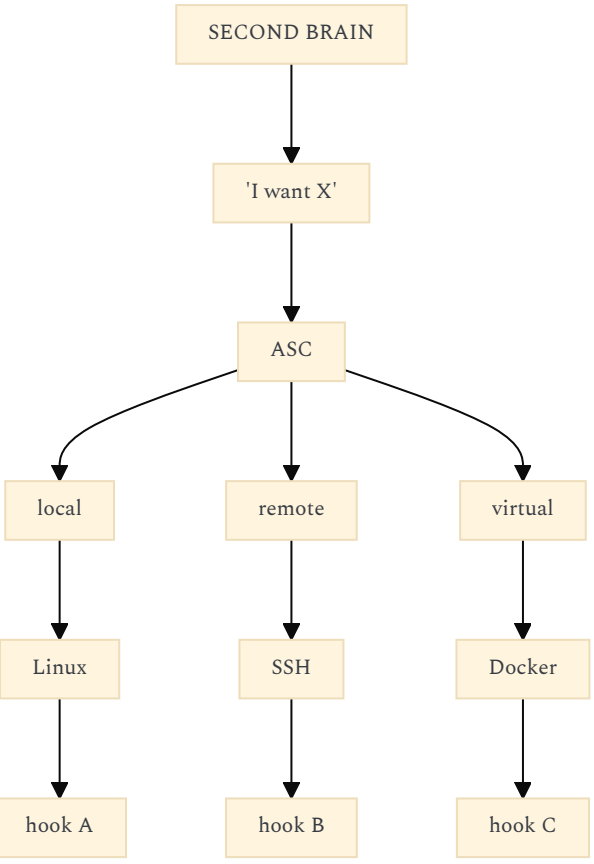
Not:



Those belong to ASC.
This is an unusually clean division of responsibility.

2. ASC BECOMES THE OPERATING-SYSTEM ABSTRACTION

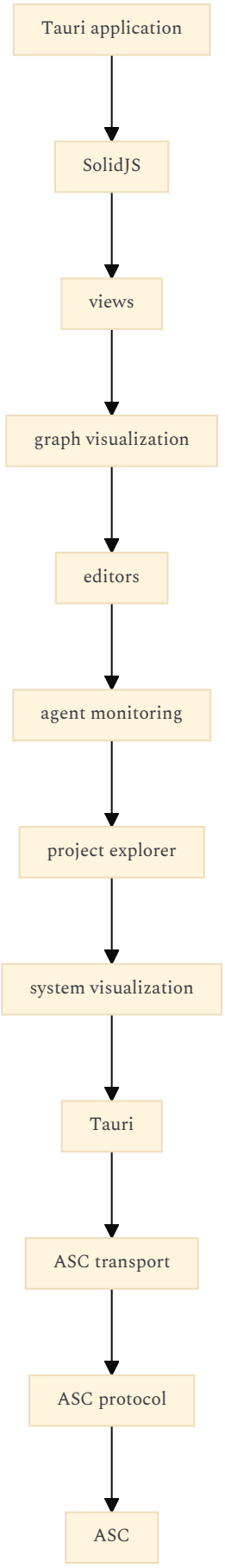
The interesting abstraction is therefore:



The UI doesn't need to know whether:
"restart service"
means:
systemctl restart foo
or:
rc-service foo restart
or:
Restart-Service foo
or eventually something else.
It asks ASC for:
restart service foo
ASC resolves that through its hook system.
That is exactly the kind of indirection that will make your application portable without polluting the frontend with platform conditionals.

3. THIS MAKES TAURI UNUSUALLY SIMPLE

The Tauri application can effectively become:



The Rust portion can consequently remain very small.
That is desirable.
You don't need to learn Rust deeply just to reproduce functionality that ASC already owns.

4. I WOULD MAKE ASC'S INTERFACE THE REAL ARCHITECTURAL BOUNDARY

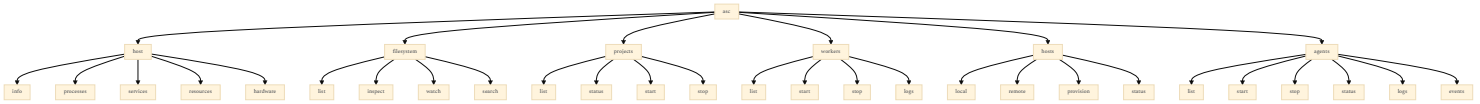
The important question is therefore no longer:

> "How should Tauri call the filesystem?"

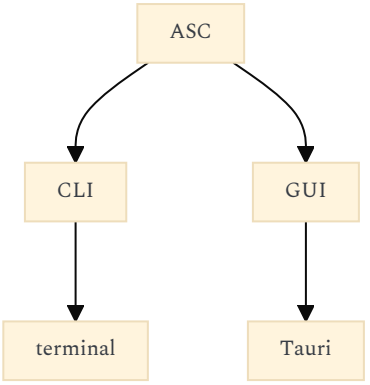
It becomes:

> "What should the ASC interface look like?"

For example:



The UI then becomes a visual client of this interface.
And because you already use ASC from the terminal, the UI and CLI become **two interfaces to the same control plane**:



This is considerably better than making the Tauri application the canonical interface.

5. THE `make` PHILOSOPHY IS ACTUALLY VERY VALUABLE HERE

Your `make` shortcuts aren't merely developer conveniences.
They can become part of the project's **operational interface**.
For example:

- make test
- make index
- make index-test
- make worker
- make graph
- make agent-test
- make provision
- make backup

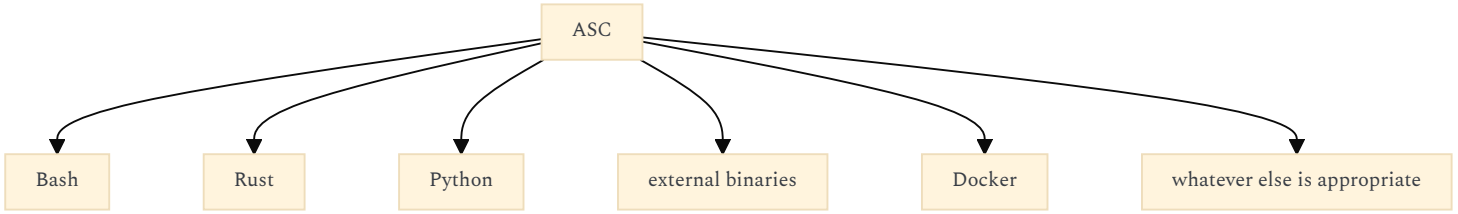
The Tauri UI can invoke the same ASC entry points that you use manually.
That produces an important invariant:

> **Anything the GUI can do should remain reproducible from the terminal.**

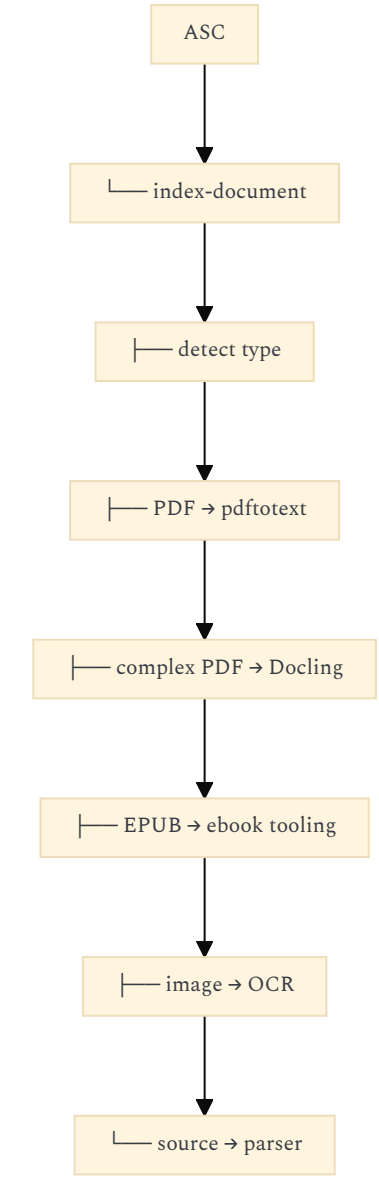
That is excellent for debugging.
Suppose the UI displays:
Indexing failed
You should be able to reproduce the operation outside the UI:
make index
or:
asc index ...
and inspect the same logs.
That eliminates an entire class of "works in the GUI but I can't reproduce it" problems.

6. YOUR LACK OF PYTHON PROFICIENCY IS NOT A PROBLEM

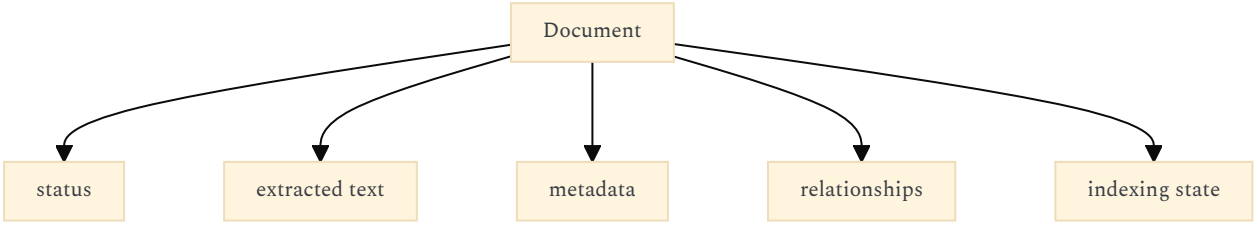
I would **not turn Python into an architectural dependency of the application**.
Instead:



ASC becomes the orchestration layer.
For example:



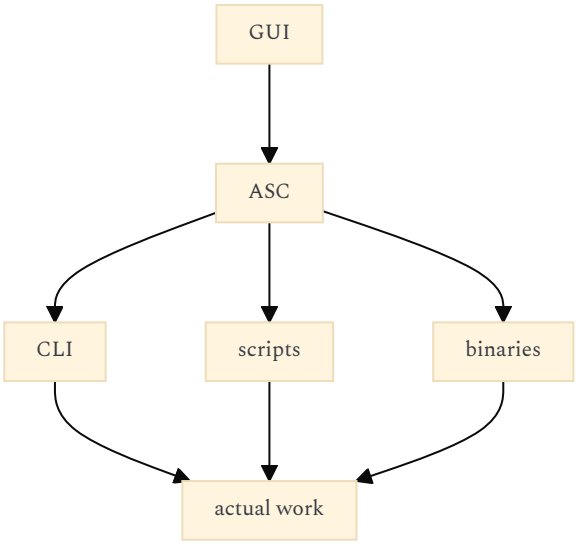
The UI doesn't care.
It sees:



That is exactly how I would handle Docling.
You don't need to become a Python developer merely because Docling happens to be written in Python.

7. IN FACT, I WOULD EMBRACE "CLI-FIRST" INFRASTRUCTURE

For your particular project, this architecture is attractive:

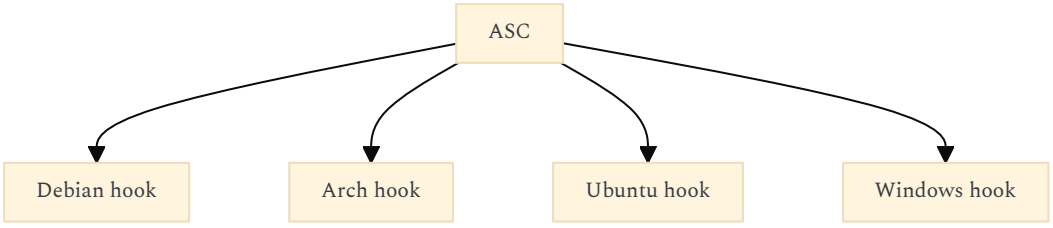


For example:
asc document extract foo.pdf
might internally execute:
python -m ...
or:
docker compose run ...
or:
docling ...
or:
pdftotext ...
The implementation can change completely while the ASC contract remains stable.
That gives you the same swappability you want at the OS level, but also at the **tool level**.

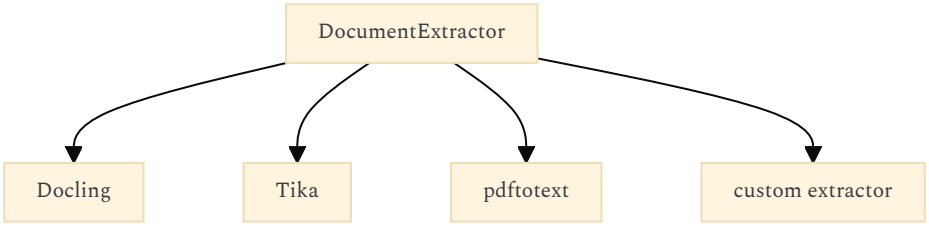
8. THIS CREATES THREE DISTINCT KINDS OF PORTABILITY

This is one of the strongest aspects of your architecture.

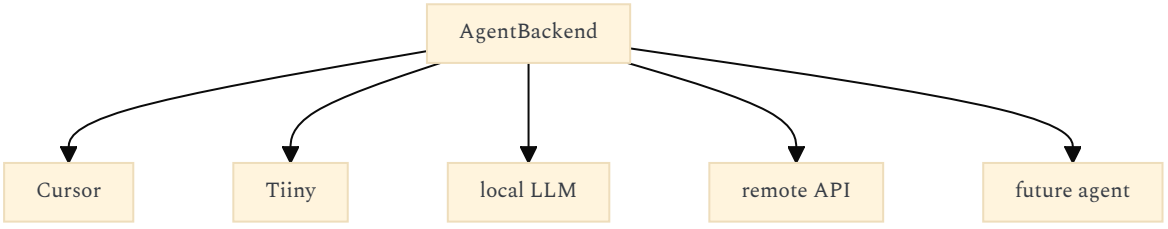
OS portability



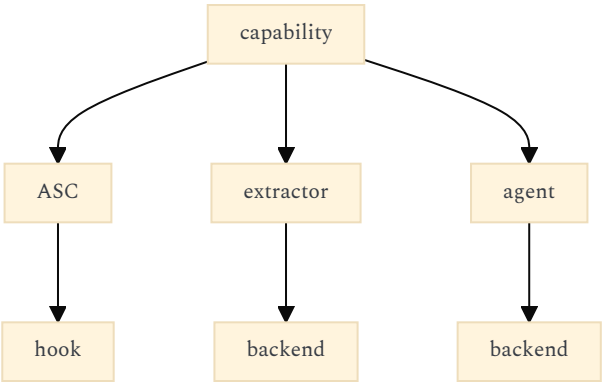
Tool portability



AI portability



The UI only sees the abstract capability.



That is much more powerful than merely making Tauri cross-platform.

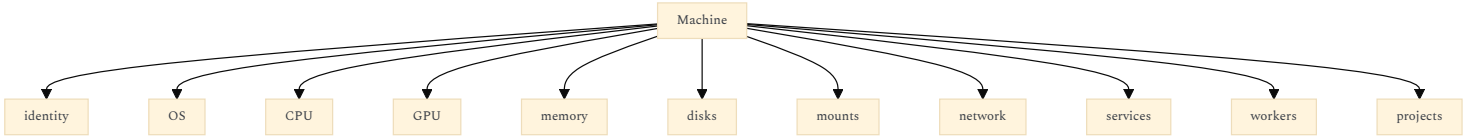
9. YOUR UI SHOULD THEREFORE BE CAPABILITY-ORIENTED

I would avoid designing the frontend around technical components such as:

- Solr page
- ArangoDB page
- Docker page
- Python page
- Tauri page

Instead:
Knowledge
- Projects
- Machines
- Storage
- Services
- Workers
- Agents
- Sources
- Graphs
- Activity

Those are stable concepts.
For example:



Whether the information comes from:

- /proc
- lsblk
- lshw
- Docker
- SSH
- Windows APIs
- ASC hooks

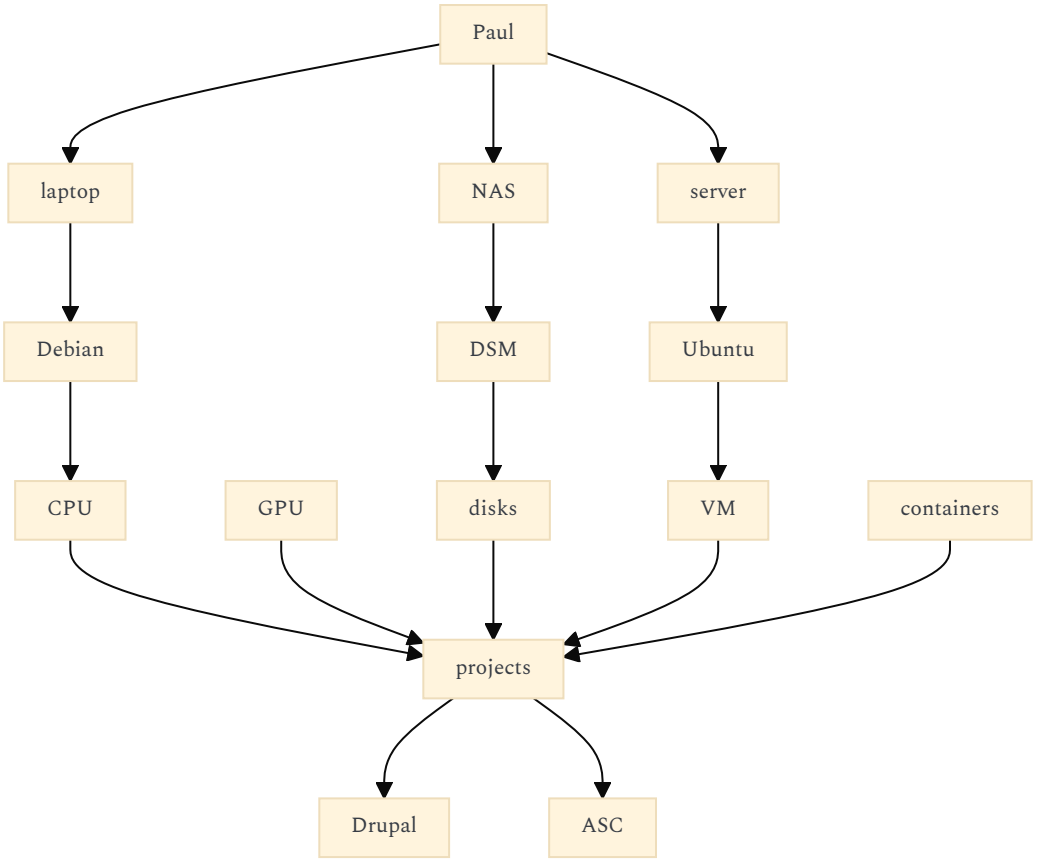
is irrelevant to the UI.

10. YOUR "HARDWARE GRAPH" IDEA FITS BEAUTIFULLY INTO THIS

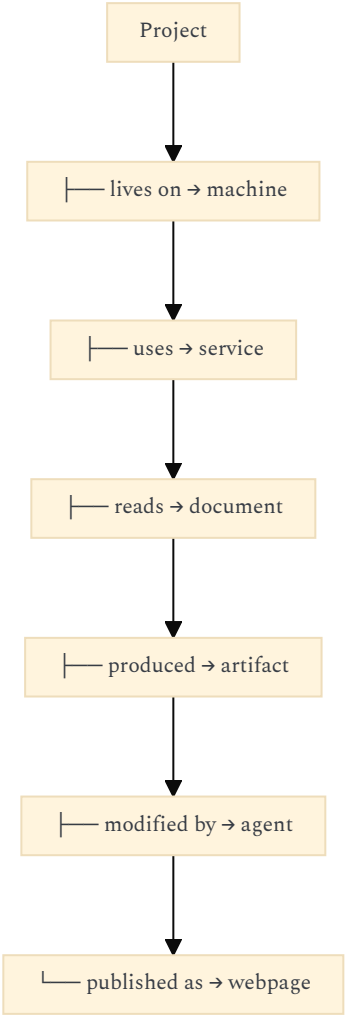
You mentioned wanting to visualize:

> the different hardware I use

I wouldn't treat that as a dashboard.
I'd treat machines as **nodes in the same knowledge graph**.
For example:



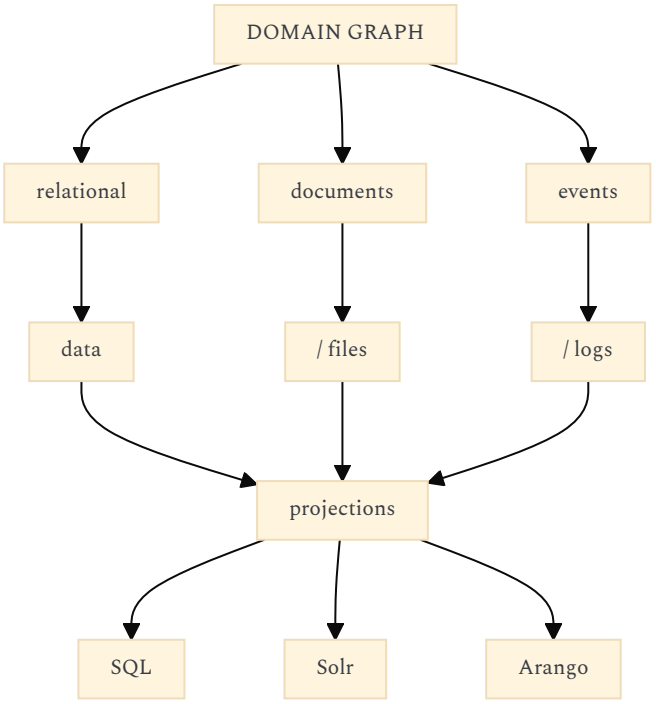
And then:



This is where the "second brain" and "ASC" aspects stop being two separate applications.
ASC supplies the **observable computational world**.
The second brain supplies the **semantic model of that world**.

11. THE GRAPH SHOULD NOT NECESSARILY BE THE DATABASE

This distinction matters.
You can represent:
Machine → Project → Document → Concept
as a graph in the UI without requiring every piece of information to live in ArangoDB.
Think in terms of:



That gives you freedom to choose storage according to workload.
The graph is the **conceptual model**, not necessarily the physical storage model.

12. KOBALTE + YOUR OWN CSS IS THE RIGHT CHOICE

Given the design philosophy you describe, I would revise my previous recommendation even more strongly:
Do not use Tailwind.
Your `chouette.net.br` work demonstrates precisely why.
You already have a coherent visual language involving:

- restrained typography
- whitespace
- explicit hierarchy
- very limited visual vocabulary
- CSS doing actual design work
- components remaining relatively transparent

That is exactly where Tailwind becomes counterproductive.
Kobalte provides the behavioral/accessibility primitives.
You provide:

- body {}
- h1 {}
- h2 {}
- p {}
- a {}
- button {}
- dialog {}
- nav {}
- article {}

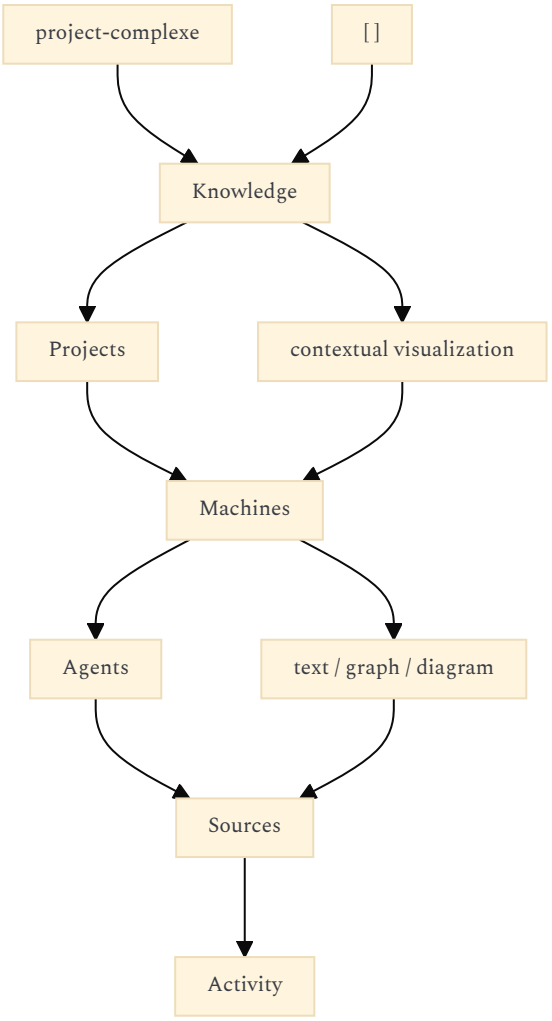
and your application-specific vocabulary:

- .knowledge {}
- .graph {}
- .machine {}
- .agent {}
- .project {}
- .source {}
- .timeline {}

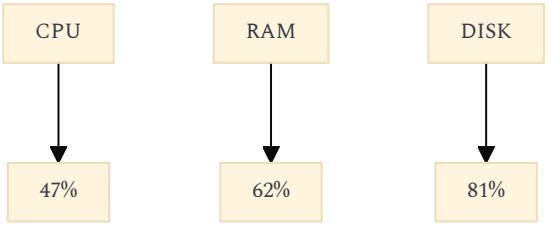
This is much closer to the kind of interface you're describing.

13. I WOULD ALSO AVOID A TRADITIONAL "DASHBOARD" AESTHETIC

Your description suggests something more interesting:



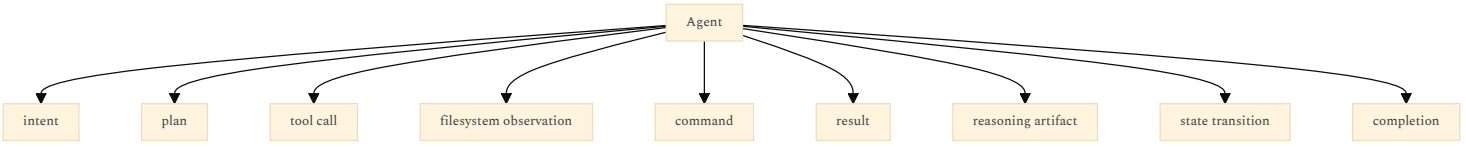
Rather than:



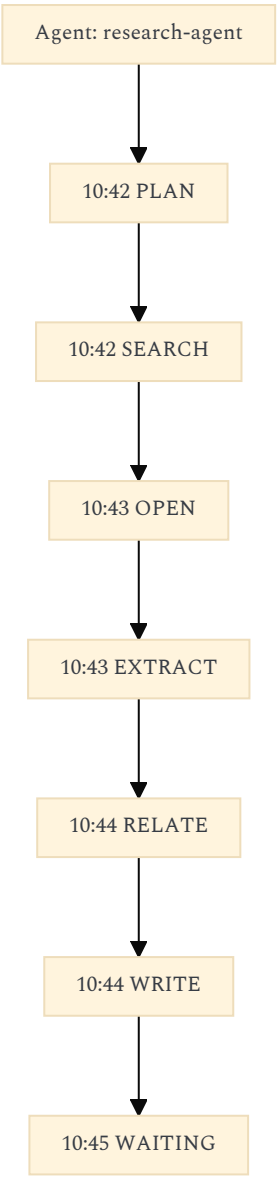
The former is much more appropriate for your goal.
The interface becomes a **visual query surface over a complex system**, not a collection of monitoring widgets.

14. AGENT MONITORING PARTICULARLY BENEFITS FROM THIS

I would represent an agent's activity as an event stream:



Then visualize it as:



The same events should be accessible through:

- GUI
- CLI
- logs
- machine-readable event stream

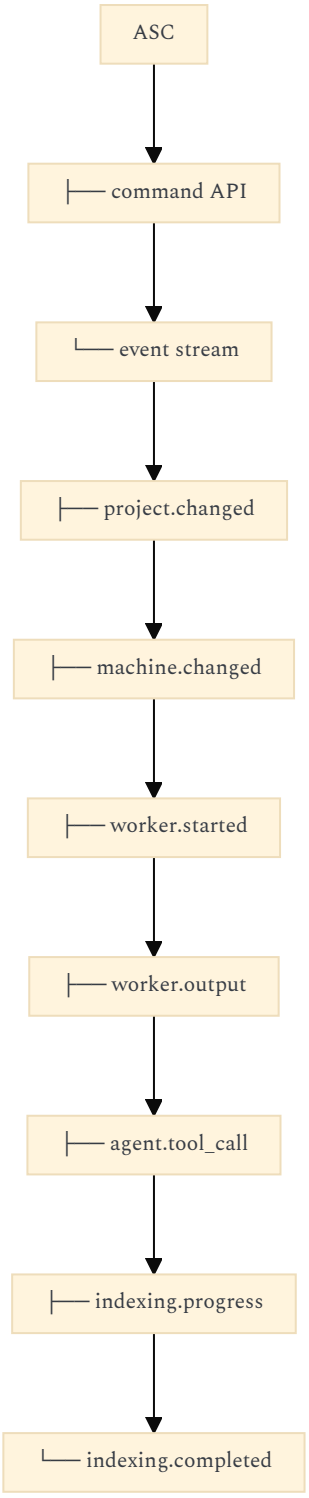
Again, ASC is an excellent place to normalize this.

15. THIS SUGGESTS AN EVENT PROTOCOL BETWEEN ASC AND TAURI

Rather than Tauri constantly polling ASC:

```
Tauri → ASC → status
Tauri → ASC → status
Tauri → ASC → status
```

I'd eventually want:



The UI subscribes.
That makes the application feel alive without turning it into a polling monster.
It also makes the CLI and future external clients possible.

16. THE MOST IMPORTANT DESIGN RULE FOR ASC

I would make this explicit very early:
ASC is authoritative about execution.
Second Brain is authoritative about interpretation.
More formally:

- ASC
- = "What exists and what can be done?"

Second Brain
= "What does it mean, how is it related, and how should I understand it?"
Examples:

ASC

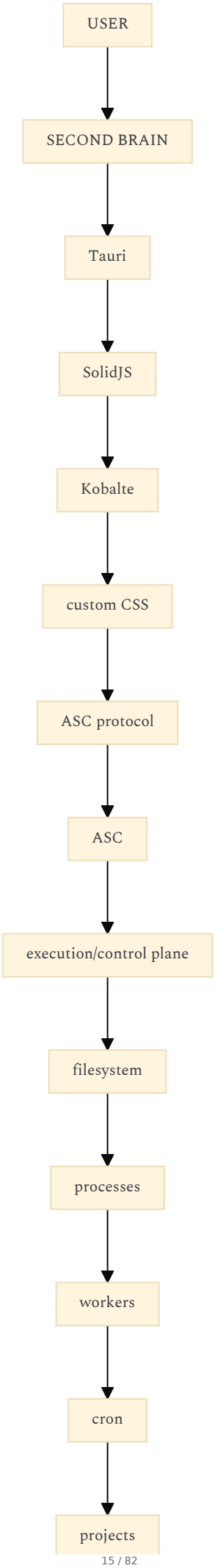
This machine has:
- 32 GB RAM

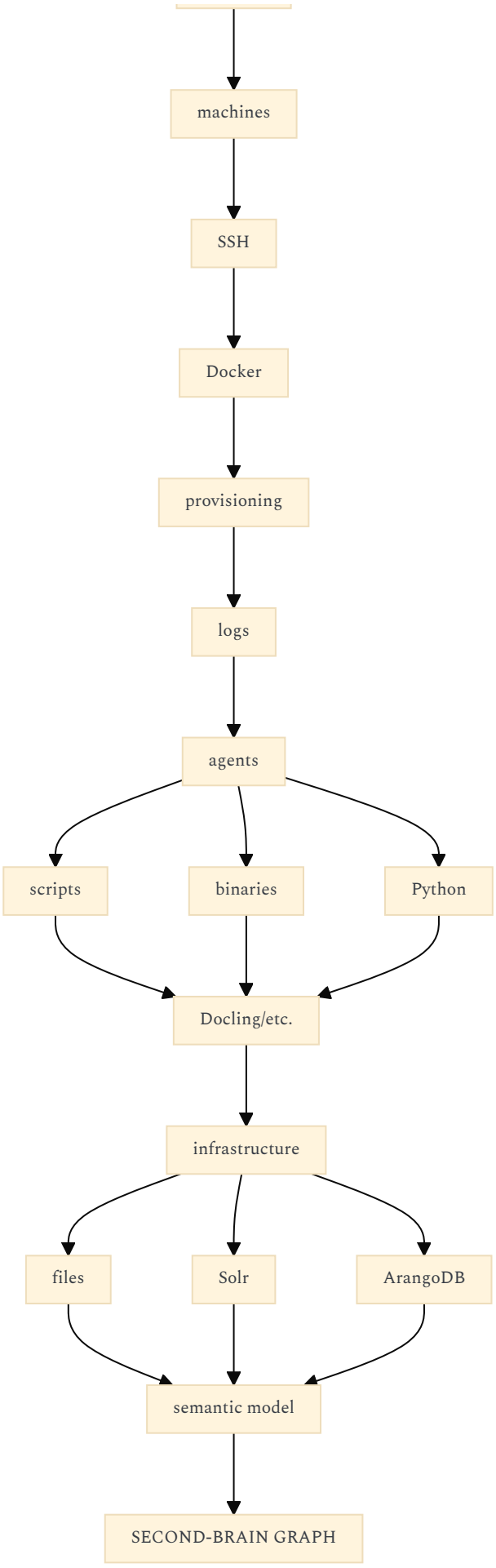
- 12 CPU threads
- GTX 1050
- Debian 13
- Docker running
- immich container running

Second Brain

This is my development laptop.
It hosts:
- ASC
- project-complexe
- Drupal project
It is related to:
- AI research
- local LLM experiments
The GTX 1050 constrains:
- local model selection
That separation is extremely powerful.

17. THE ARCHITECTURE I'D NOW RECOMMEND


USER
SECOND BRAIN
Tauri
SolidJS
Kobalte
custom CSS
ASC protocol
ASC
execution/control plane
filesystem
processes
workers
cron
projects



The apparent circularity is intentional: ASC exposes the computational world; the second brain builds a semantic representation of it.

18. CONSEQUENTLY, MY REVISED STARTER RECOMMENDATION IS EVEN SIMPLER

I would now **not use** `tauri-start-solid` **as the foundation**.
I'd start with:

- Tauri 2

- SolidJS
- TypeScript
- Vite
- Kobalte
- plain CSS

Nothing else.
Then add:
TanStack Router
when you actually need routing.
Possibly:
TanStack Query
when ASC becomes an actual query/event backend.
And otherwise keep the dependency tree aggressively small.
Your project's differentiator isn't going to be its component library.
It will be:

```
       ASC
        +
  knowledge model
        +
     indexing
        +
      graph
        +
agent orchestration
        +
  visualization
```

The frontend should therefore be almost boring technologically.
That is a good thing.

19. ONE ARCHITECTURAL CONSEQUENCE I WOULD TAKE VERY SERIOUSLY

Don't make the first milestone:

> "Build the second-brain interface."

Make it:

> "**Make ASC expose a stable machine/project/event interface that the UI can consume.**"

Then the first Tauri screen can be almost absurdly simple:

- Machine
- Debian 13
- 12 CPU threads
- 32 GB RAM
- GTX 1050
- ASC
- running
- Projects
- projet-complexe
- asc
- chouette.net.br
- Workers
- indexing idle
- backups running

If that works from:
terminal → ASC
and:
Tauri → ASC
you have established the most important architectural invariant.
Everything else can grow around it.
Confidence: 0.97. Your clarification makes the ASC-as-control-plane boundary unusually well-defined. The strongest architectural risk is not Tauri, SolidJS, Kobalte, or Python; it is allowing the second-brain UI to acquire direct knowledge of OS/tool implementations that ASC is specifically intended to abstract. Keeping that boundary strict preserves the portability, terminal reproducibility, lightweight frontend, and eventual agent swappability you are aiming for.

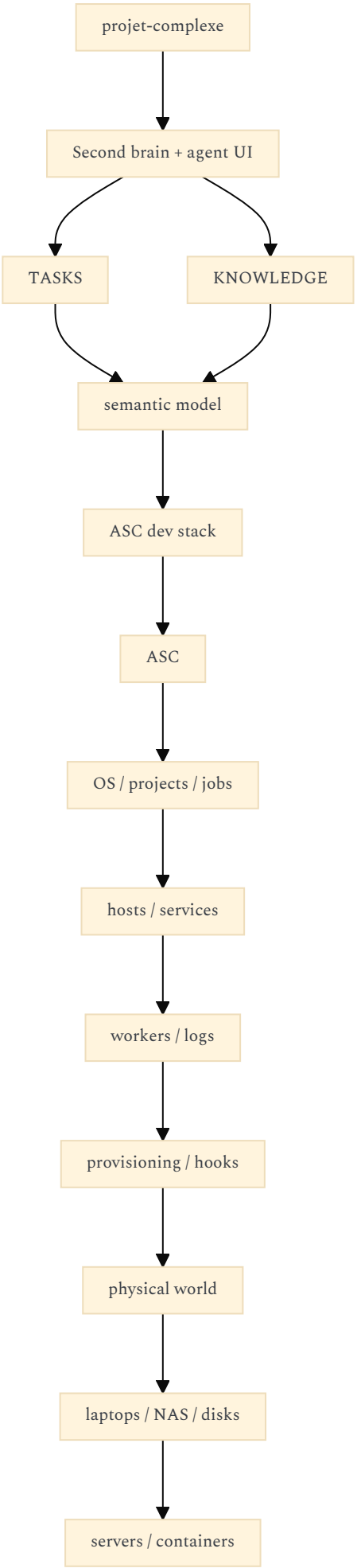
FOLLOW-UP 1 : WHERE TO DRAW THE LINE BETWEEN PROJECTS (RESPONSIBILITIES)

What I liked about the https://github.com/Paulmicha/projet-complexe project was its separation (visually) between **task-oriented** and **knowledge-oriented** use cases. The https://github.com/Paulmicha/asc is much more generic, but I figured https://github.com/Paulmicha/projet-complexe could just rely on it. So I

am torn between :

- either make Paulmicha/projet-complexee a monolithic repo using ASC, or
- having a separate "dev stack" repo using Paulmicha/ascc that would be the underlying layer for the Paulmicha/projet-complexe (which is no only a UI but could evolve into some sophisticated second brain and AI agents controller)

I think I prefer the second option. What do you think ?
https://chatgpt.com/s/t_6a7cd31e80c881919007c17673f2739a
I agree with your second option, with one refinement:
I would keep three conceptual layers, but only two application repositories:



The important point is that `projet-complexe` **should not become a generic system-management application merely because ASC can expose all that information**.
Its original distinction between **task-oriented** and **knowledge-oriented** work is valuable. I would preserve it as a fundamental UX and domain distinction.
The "dev stack" should be the thing that makes ASC useful to the second brain, not something that gets absorbed into it.

Confidence: 0.96.

1. WHY I THINK THE SEPARATE DEV-STACK REPOSITORY IS THE BETTER ABSTRACTION

The two projects have different reasons for existing.

ASC answers:

> How do I operate my computational environment?

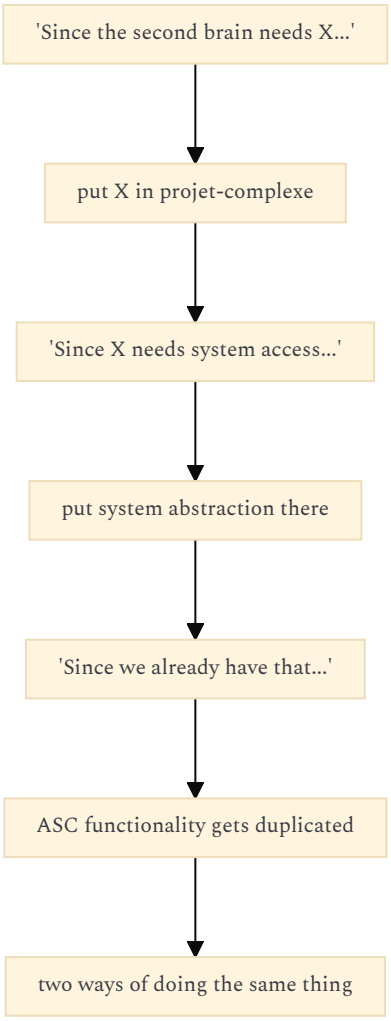
- machine
- filesystem
- process
- worker
- service
- container
- remote host
- package
- cron
- project
- backup
- provisioning
- logs

`projet-complexe` answers:

> How do I understand, organize, manipulate and act upon what exists in that environment?

- task
- project
- idea
- document
- source
- concept
- research
- relationship
- agent
- plan
- publication
- knowledge

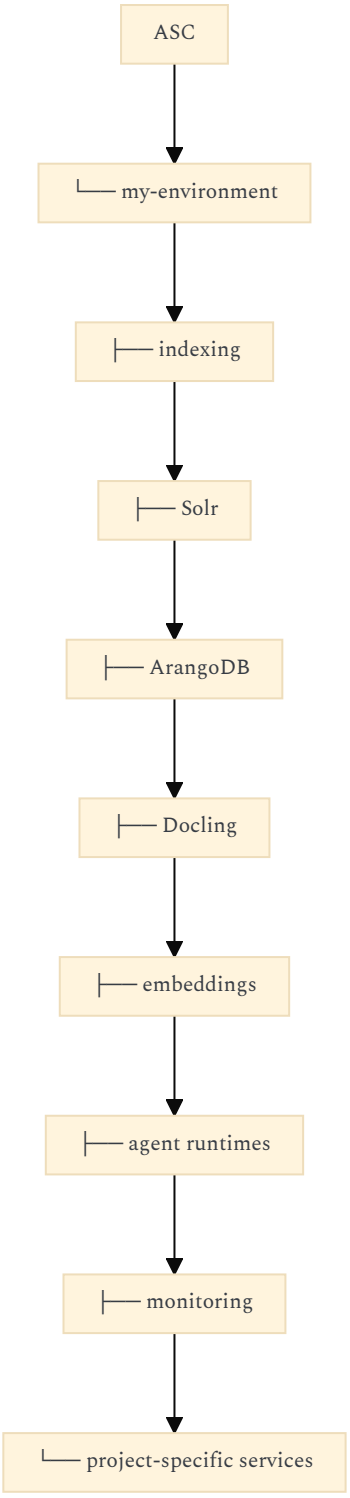
These are related, but they are **not the same domain**.
Trying to put them into one repository would create a dangerous gravitational pull:



The separate repository creates a hard architectural constraint against that drift.

2. BUT I WOULD NOT CALL IT SIMPLY A "DEV STACK"

I think there is a subtle naming issue.
What you're describing is more than:
development environment
It is becoming a **local runtime/environment configuration for your personal computing ecosystem**.
Something like:
asc-environment
or:
asc-stack
or:
asc-workspace
would make more sense to me.
Its job would be to instantiate a particular **environment around ASC**:



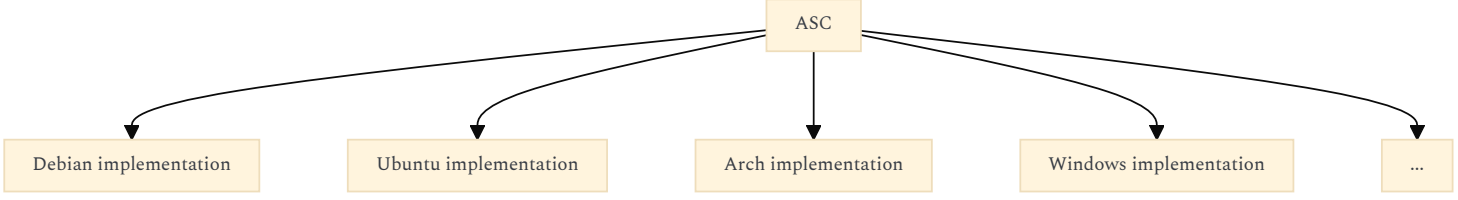
That repository can then contain the things that are *specific to your second-brain environment* without polluting ASC itself.

3. THIS GIVES YOU THREE DIFFERENT KINDS OF PORTABILITY

This is where the separation becomes particularly powerful.

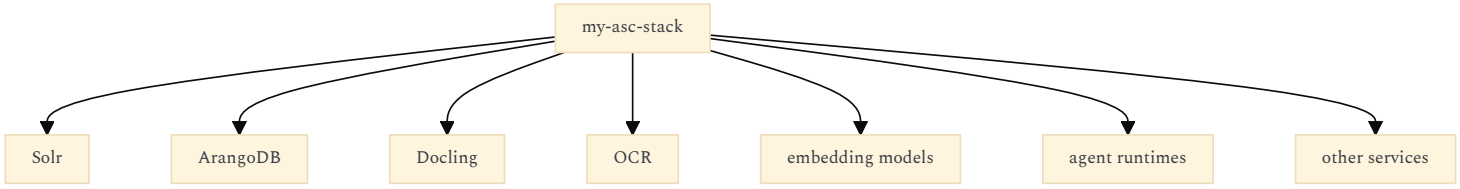
ASC itself

Portable:



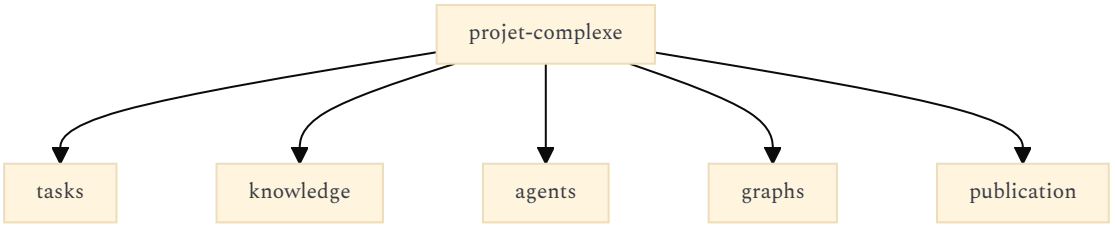
Your ASC environment

Personal / machine-specific:

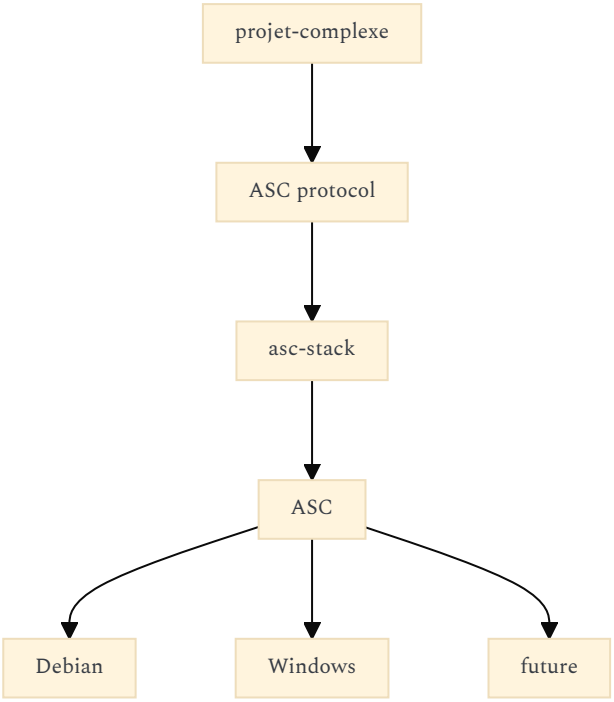


The second brain

Conceptually portable:



This gives you:



The second brain doesn't need to know whether the environment underneath it is your Debian laptop or some future Windows machine.

4. MORE IMPORTANTLY: DON'T MAKE `projet-complexe` DEPEND ON YOUR ENTIRE DEV STACK

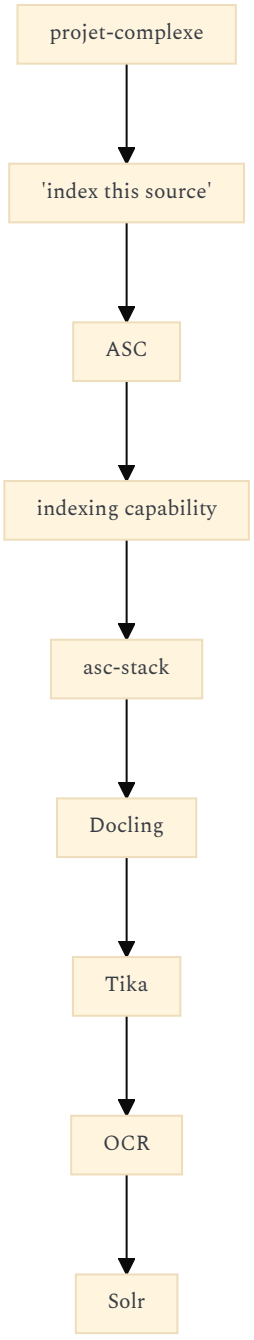
I'd distinguish:
ASC capability
from:
my ASC environment capability
For example, `projet-complexe` might ask:
ASC:

```
search documents
```

It should **not** ask:
ASC:

```
run the Solr Docker container
execute Docling
query Solr directly
```

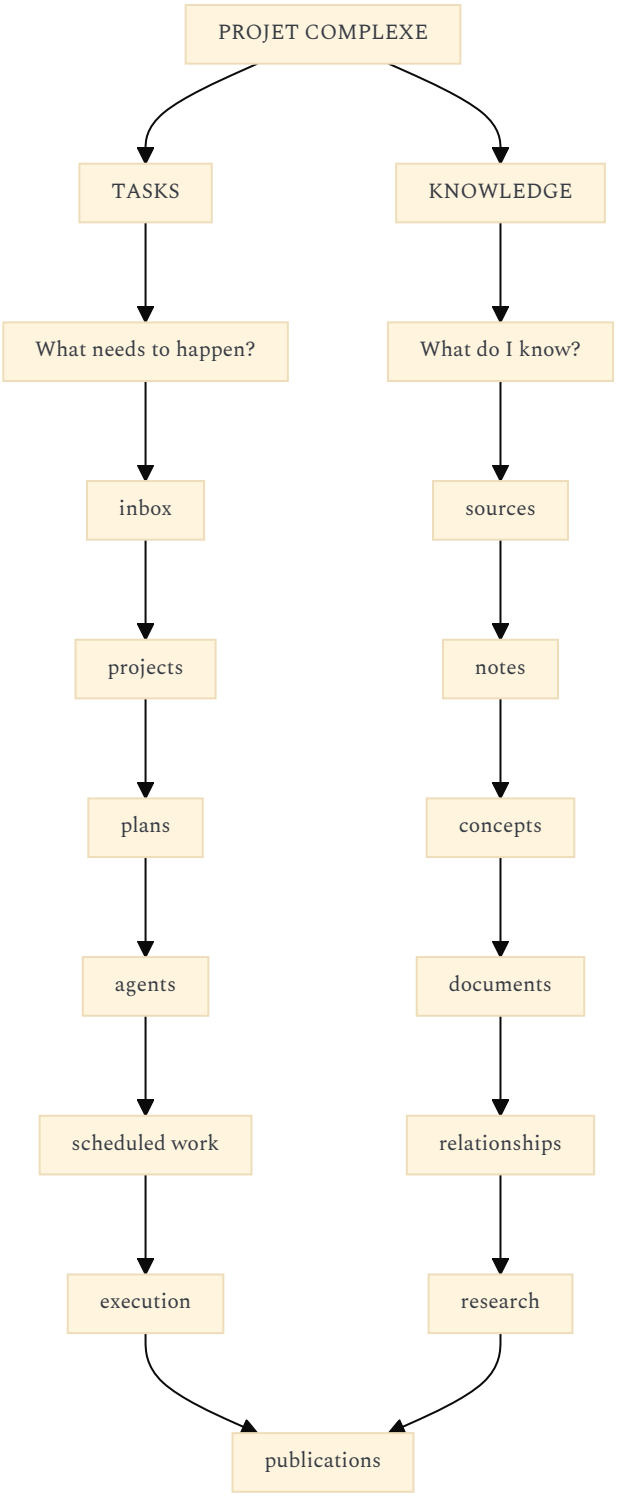
The latter is environment implementation.
Conceptually:



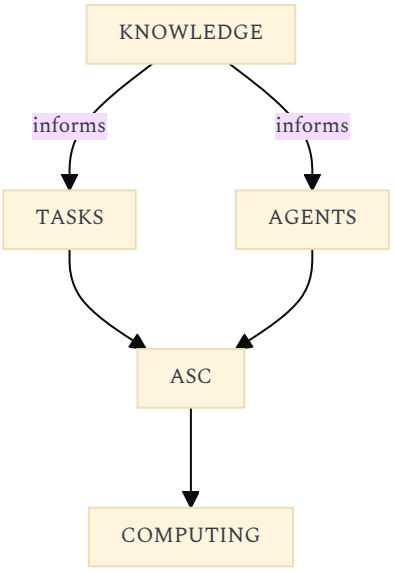
That distinction will save you a lot of coupling later.

5. THE VISUAL DISTINCTION YOU LIKED SHOULD SURVIVE ALMOST UNTOUCHED

This is actually the strongest argument for keeping `projet-complexe` independent.
The old application's conceptual split can become something much richer:



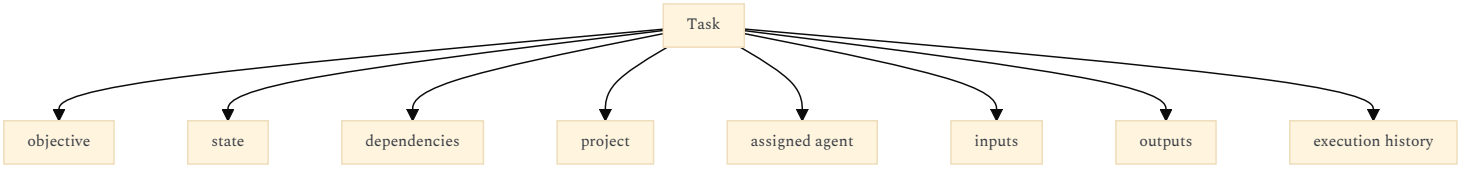
Then there is a third dimension:



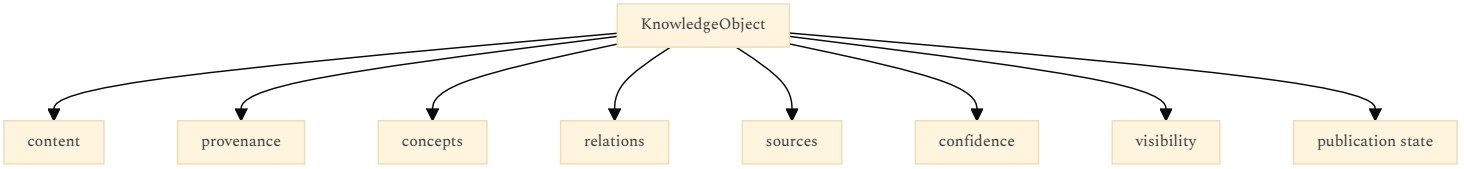
That is much more interesting than a conventional task manager + note-taking application.

6. I WOULD ACTUALLY AVOID "TASK VS KNOWLEDGE" BEING MERELY NAVIGATION

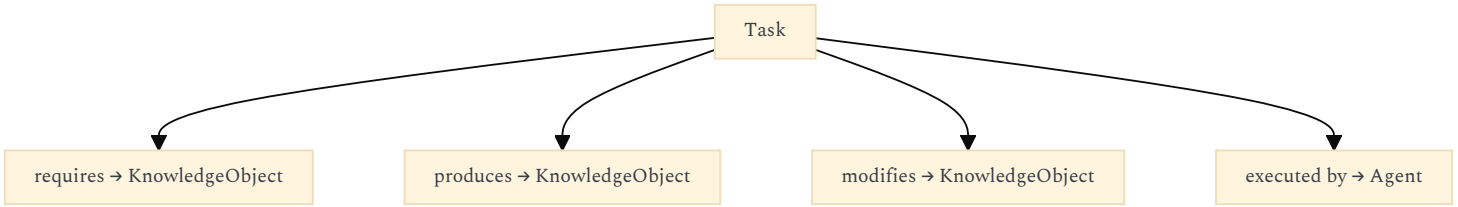
Make it a **domain-level distinction**.
For example:



versus:



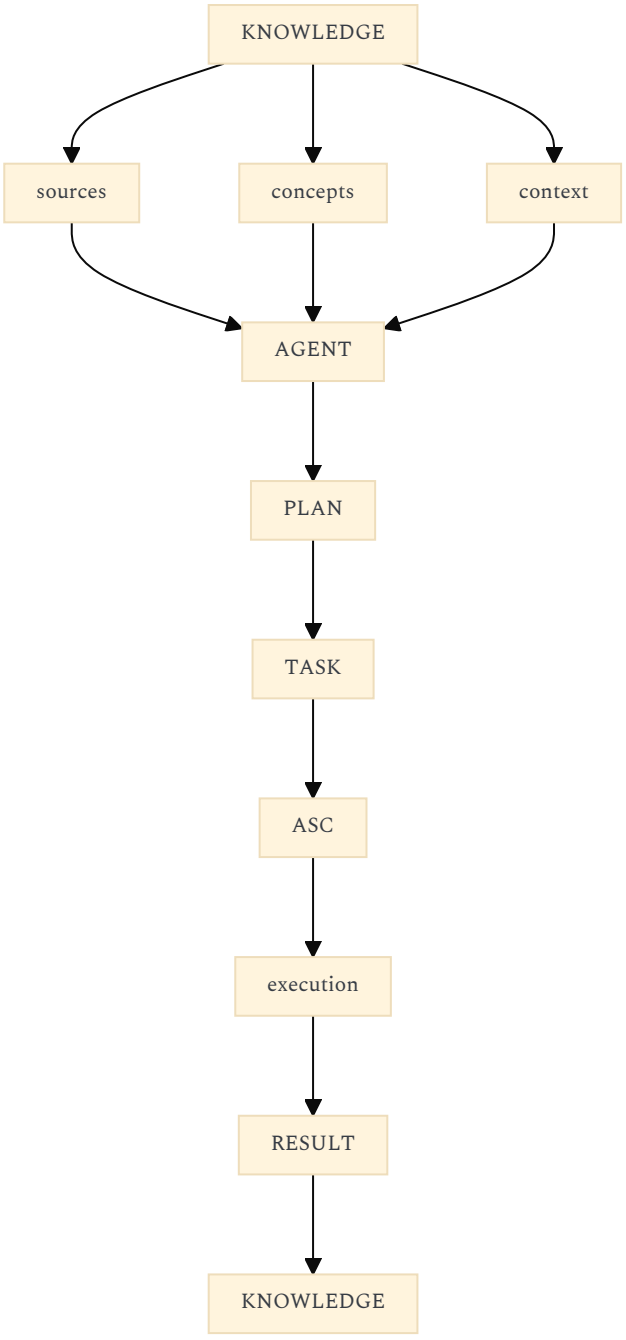
They can then intersect:



That is a much stronger model than:
tasks/
notes/

7. AND THIS GIVES AGENTS A VERY NATURAL POSITION

Agents don't need to be another top-level information silo.
They become the **bridge between task and knowledge**.



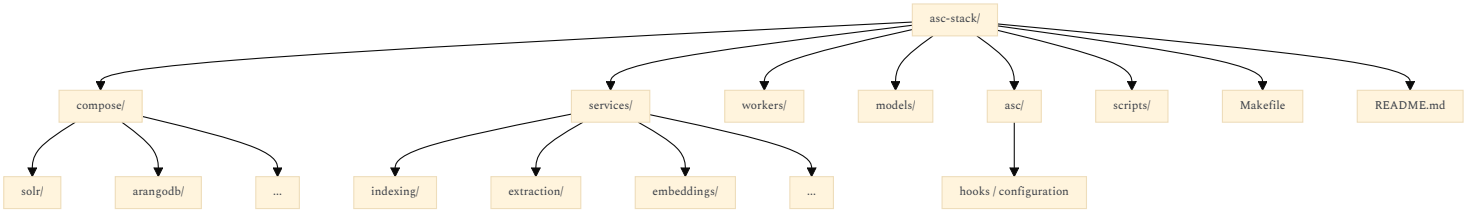
This is where your earlier interest in agent autonomy becomes relevant.
The agent isn't simply:

> "an LLM with tools."

It becomes a process moving through your **task/knowledge environment**, while ASC provides the actual operational capabilities.

8. THE DEV STACK THEN BECOMES AN IMPLEMENTATION SUBSTRATE

I would expect something like:



But importantly:
the second brain doesn't import this repository.
It talks to ASC.
The dev stack configures ASC so that the required capabilities exist.

9. THIS ALSO FITS YOUR "EVERYTHING EVENTUALLY BECOMES VISUAL" AMBITION

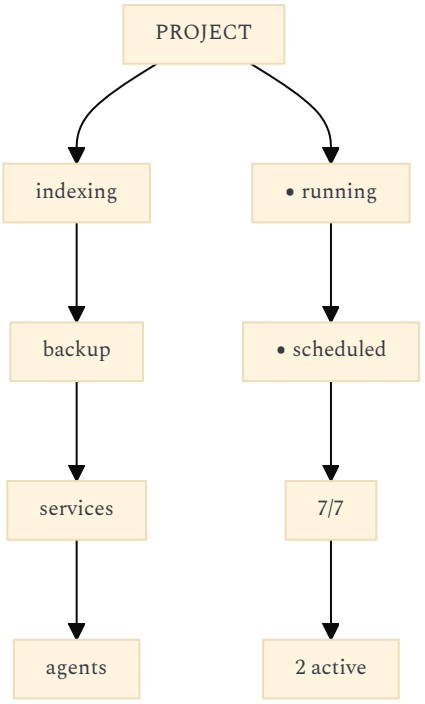
You said something important:

> gradually move the things I currently implement and run manually in ASC ... into their corresponding visual representation in the UI

That means you don't actually want to replace ASC.
You want to **visualize ASC**.
For example, today:

- make backup
- make index
- make restart-service
- make project-status

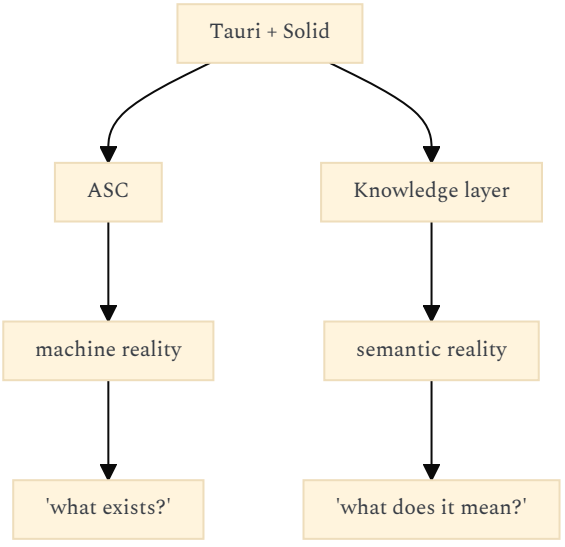
Tomorrow:



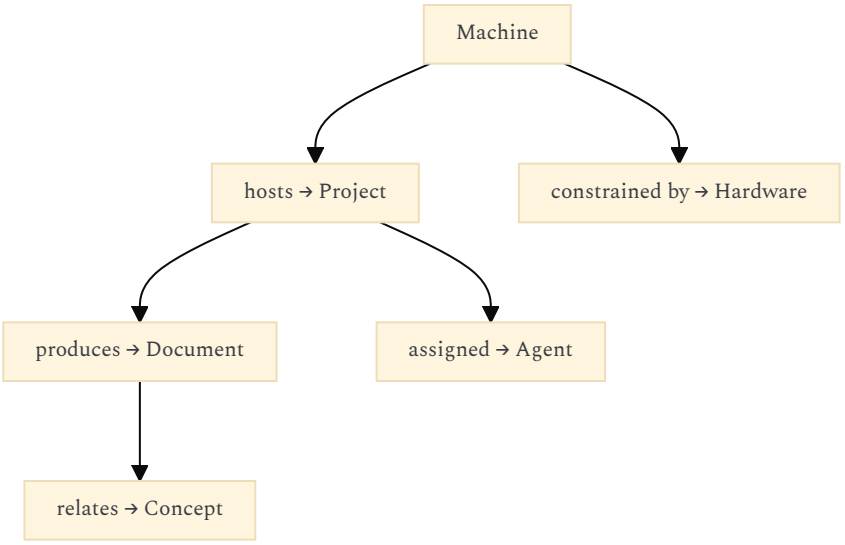
But both ultimately invoke the same ASC capabilities.
That's excellent because the GUI isn't creating a parallel operational reality.

10. THE UI BECOMES AN ASC CLIENT PLUS A KNOWLEDGE SYSTEM

I would therefore think of the Tauri application as having **two distinct backends**:



And the really interesting functionality emerges from their intersection:



That is the "second brain" part.

11. I WOULD THEREFORE KEEP `projet-complexe` SURPRISINGLY IGNORANT OF ASC INTERNALS

For example, avoid frontend concepts like:

- SolrService
- ArangoService
- DockerService
- LinuxService
- PythonService

Prefer:

- Search
- Indexer
- Graph
- Machine
- Project
- Worker
- Agent
- Storage
- Task
- Source

And then let ASC provide those capabilities.
This means that if six months from now you decide:

> Solr was a mistake; let's use something else.

`projet-complexe` shouldn't care.
Likewise:

> Docling is too heavy; use Tika + custom parsers for 80% of files.

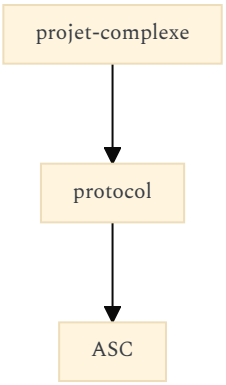
No UI redesign.
And:

> I want to run agents through Tiiny instead of Cursor.

No conceptual change.
That's exactly the kind of **architectural plasticity** your project seems to be aiming for.

12. THERE IS ONE THING I WOULD PUT IN A THIRD REPOSITORY EVENTUALLY

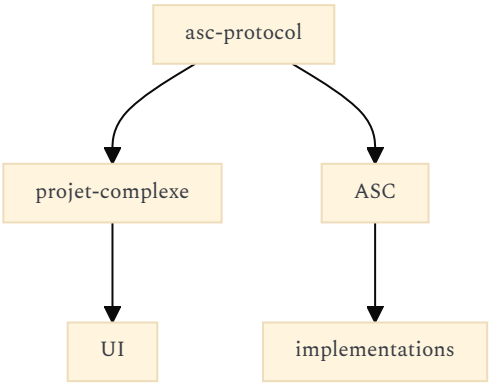
Not now, necessarily.
But conceptually there is a useful future boundary:



That protocol could eventually deserve its own small package/specification:
asc-protocol
containing things such as:

- commands
- events
- capabilities
- schemas
- identifiers
- error formats
- authentication

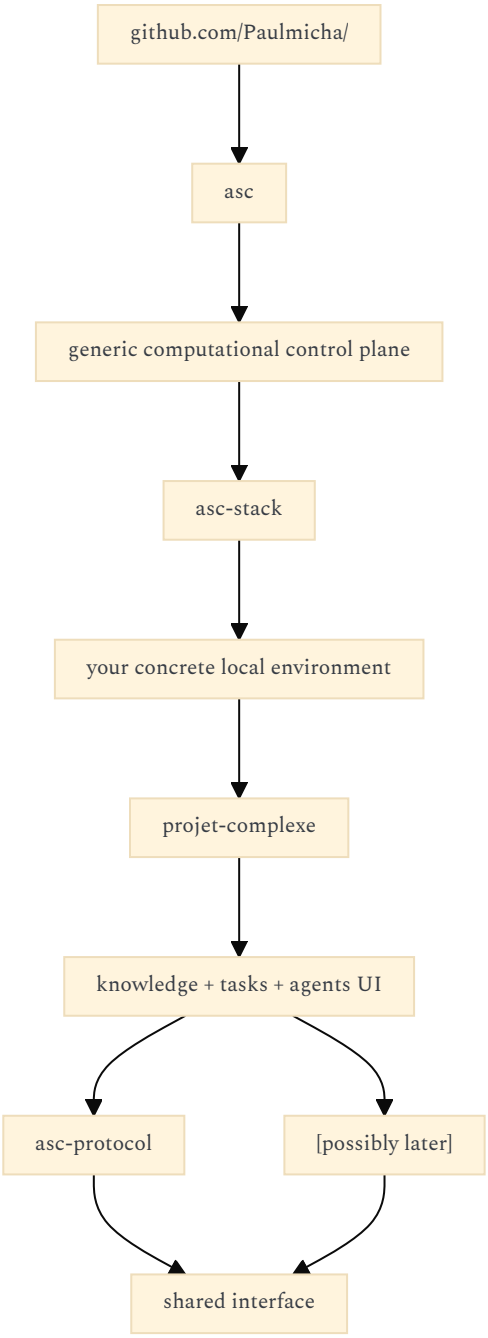
Then:



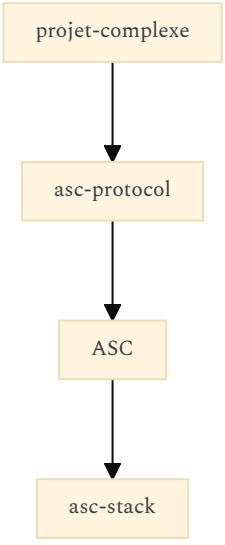
I would **not create this repository prematurely**. First let the interface emerge from actual use.
But keeping the possibility in mind will prevent you from accidentally making Tauri ↔ ASC communication proprietary to the UI.

13. THE RESULTING REPOSITORY ECOSYSTEM IS QUITE ELEGANT

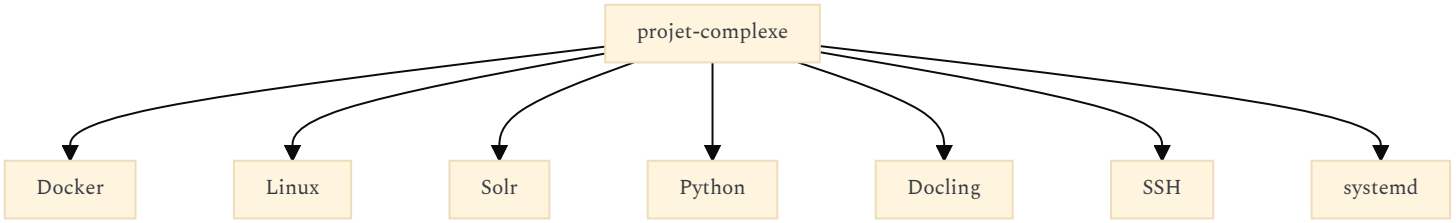
Eventually I could see your personal ecosystem becoming:



With the dependencies:



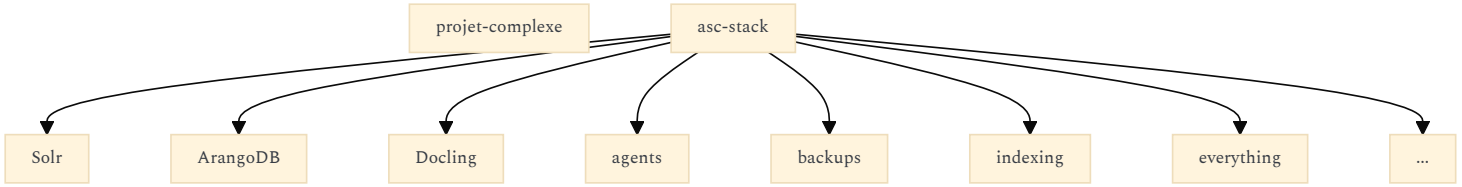
Notice what this prevents:



That would be exactly the wrong direction.

14. ONE CAVEAT: DON'T LET THE "DEV STACK" BECOME A SECOND MONOLITH

There is one architectural danger in your preferred option.
You could simply move the monolith one level down:



and eventually `asc-stack` becomes an enormous personal infrastructure repository.
I would resist that.
The distinction should be:
ASC
= generic capabilities
asc-stack
= composition/configuration of capabilities
projet-complexe
= semantics + interaction + visualization
That's enough.

15. THE DEEPEST REASON I THINK YOUR SECOND OPTION IS CORRECT

It preserves **three different kinds of identity**.

ASC

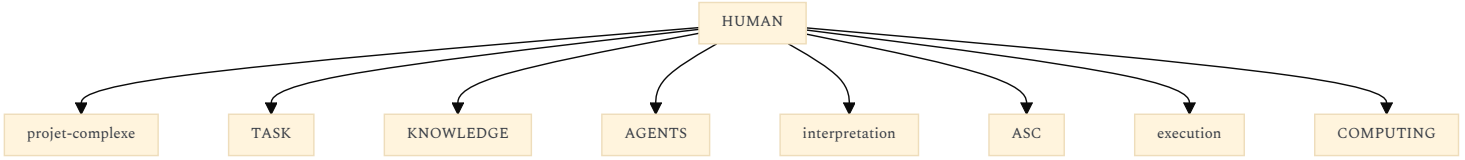
How things are operated.

`asc-stack`

How your particular computational environment is assembled.

`projet-complexe`

How you understand and work with that environment.
Those are three genuinely different concerns.
And your original task/knowledge distinction gives `projet-complexe` an additional identity that neither ASC nor the dev stack possesses:



That is a strong architecture.
I would therefore **keep** `projet-complexe` **as a separate repository and evolve it into the second-brain/agent environment**, while making an `asc-stack`-type repository the concrete infrastructure composition underneath it. The visual task/knowledge split should remain one of the defining architectural concepts of `projet-complexe`, not be swallowed by ASC. **Confidence: 0.97.**

FOLLOW-UP 2 : EMBED THE ASC-PROTOCOL IDEA INTO ASC ITSELF (JUST USE YAML)

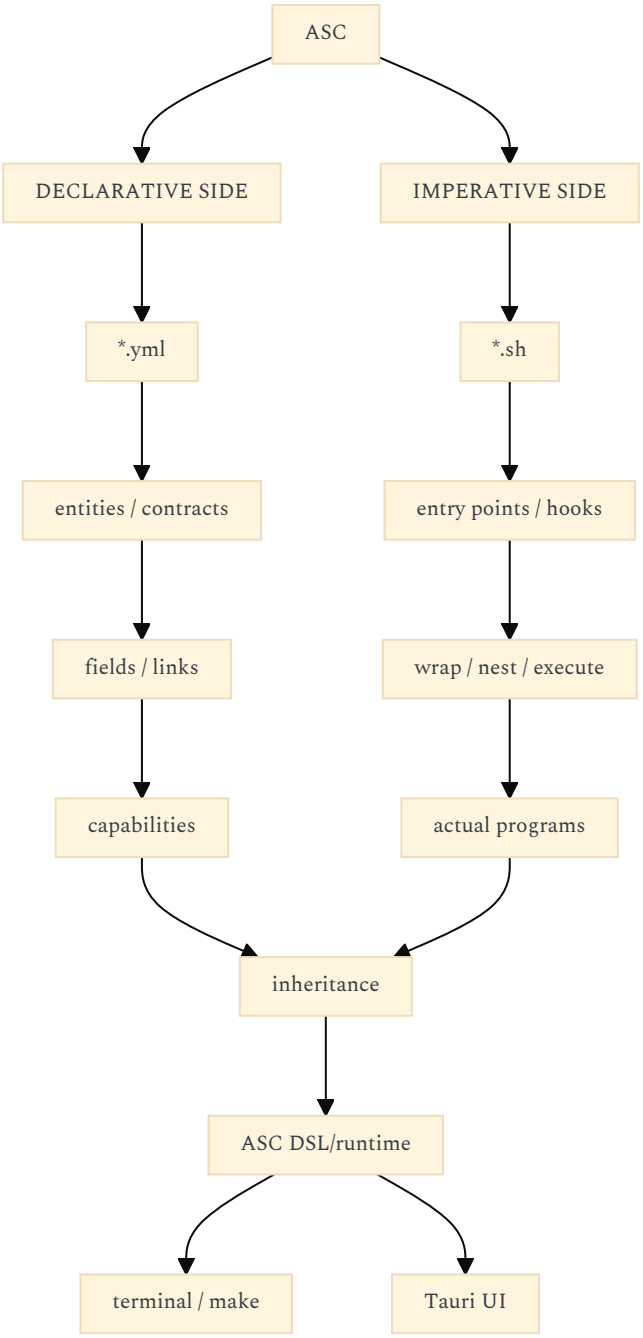
About "asc-protocol" :
Inside ASC itself, instead of a separate "asc-protocol" project, I am planning on using yml files declarations - like in the stubs I pushed over the past few weeks in https://github.com/Paulmicha/asc (still not implemented, but planned), e.g. look at :

- abstract entity declaration stub : https://github.com/Paulmicha/asc/blob/main/asc/extensions/entity/entity/entity.entity.yml
- concrete entity declaration example (stub) : https://github.com/Paulmicha/asc/blob/main/asc/extensions/workflow/idea/idea.entity.yml

This is not fixed yet, but it must allow to specify yml props that can be used by any entity eventually inheriting from it. For example, if we declare a generic "software.entity.yml", its descendants might have fields like "status" (installed, (un)healthy, out of date...), "file(s)" (executable binary or script, storage) and/or "dir(s)" (install path, assets,) sidecar(s), and dependencies.
If we take an example representing an action like OCR :

- abstract entry point provided by ASC core : https://github.com/Paulmicha/asc/blob/main/asc/extensions/cognition/recognize/text.sh
- concrete (hook) implementation : https://github.com/Paulmicha/asc/blob/main/scripts/asc/contrib/asc/tesseract/recognize/text.hook.sh

In this example, "tesseract" could be declared as an entity inheriting from software.entity (via an "include" root prop in the yml file).
Another possibility of using yml files could be to have "*.able.yml" files (called "contracts") that could act like interfaces in the object-oriented paradigm. They, too, could be "used" via include.
A yml "include" would simply merge the props, fields, etc. Obviously this is meant to be extremely minimal, and ASC "core" would only support a small subset of the Yaml specification (limited by https://github.com/Paulmicha/asc/blob/main/asc/yml/yml.inc.sh)
Essentially, the goal for ASC would be to represent anything that can be done in the shell as a custom DSL. Basic example would be :
test-in(foo,[bar,baz])
That would be equal to calling this in a terminal :
make test-in foo "'bar baz'"
or :
asc/utils/test/in.sh foo 'bar baz'
Or even :
make 'test-in(foo,[bar,baz])'
So this custom ASC DSL could be called from the Tauri project, like any other make entry point (and make could be made so understand the DSL as a fallback if none other entry point matches).
The remaining things to figure out would be the argument mapping and things like wrapping (any program called in terminal) VS "subshelling" (nesting ?) when we would want to run chains or parallel batches of programs.
And arguments mapping, with a naming convention that must remain "filename safe".
https://chatgpt.com/s/t_6a7cfed504448191b11083479cc8962d
–
Yes. With this clarification, I would **drop the idea of a separate `asc-protocol` repository entirely**.
What you are designing inside ASC is actually more interesting: **the YAML declarations themselves become the protocol/schema layer, while the shell entry points become the executable semantics**.
The architecture starts looking like this:



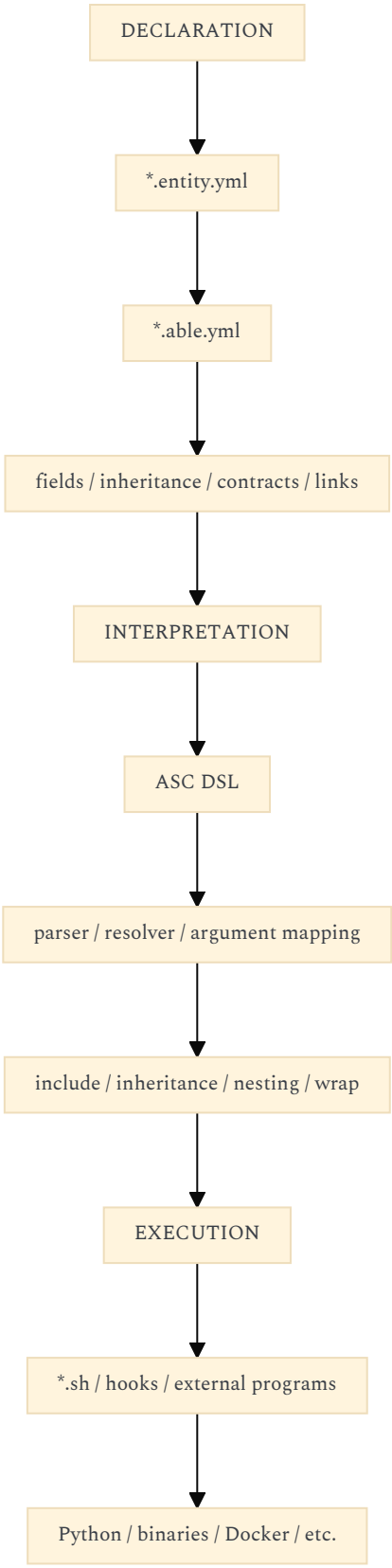
I think this is a **very good direction**, provided you keep one distinction extremely clear:

> **YAML declares what something is/can do; the shell layer defines how it actually does it.**

Your current `entity.entity.yml` already points in this direction: it treats things such as `field`, `triple`, `link`, `hook`, `wrap`, `nest`, `sidecar`, `contract`, `required`, and `optional` as part of the vocabulary ASC itself understands. (GitHub)
And your OCR example is almost a perfect demonstration of the complementary imperative layer: the abstract `recognize/text.sh` establishes the operation and invokes pre/main/post hooks, while the Tesseract hook supplies one concrete implementation. (GitHub)
Confidence: 0.97.

1. I THINK THERE ARE ACTUALLY THREE LAYERS IN ASC

Not two.



This is important because it prevents YAML from gradually becoming an imperative programming language.
I would resist adding things like:

- run:
- command: ...
- if: ...
- foreach: ...
- parallel: ...

too early.
The more powerful model is:

- software:
- ...

plus:

- software/install

- software/status
- software/uninstall

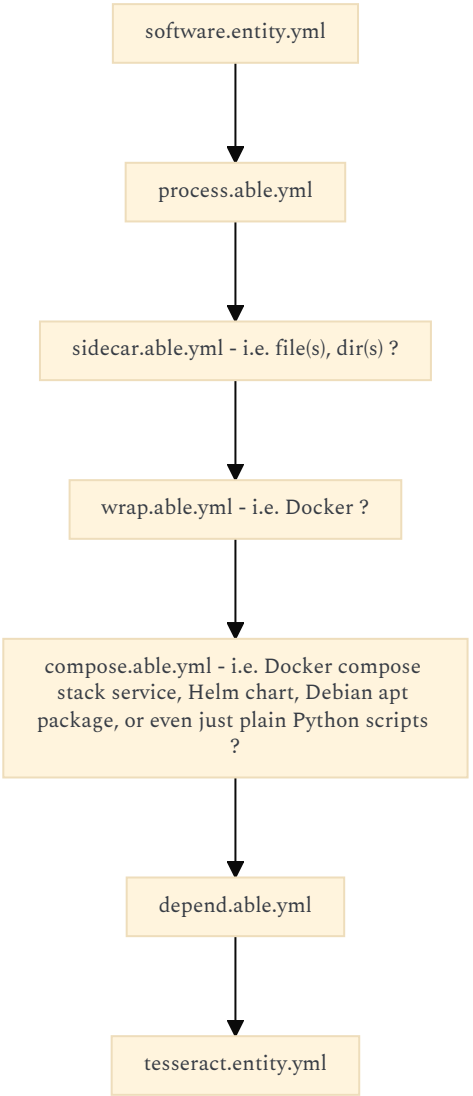
plus hooks that resolve those operations.
The YAML remains **descriptive**.
The DSL remains **compositional**.
The shell remains **executable**.

2. YOUR `include` IDEA IS PARTICULARLY IMPORTANT

Your current root entity already has:
include: '*'
and explicitly associates `include` with inheritance/dependencies. (GitHub)
The `idea.entity.yml` stub then demonstrates the very lightweight form you're aiming for:
include:
- sidecar.able
(GitHub)
I think this is exactly the right instinct.
But I would make one conceptual distinction now:
include
doesn't necessarily mean classical inheritance.
It means:

> **compose this declaration with these other declarations.**

That gives you much more freedom.
For example:



The result is a composed declaration.
That avoids having to reproduce the entire complexity of OO inheritance.

3. YOUR `software` EXAMPLE DEMONSTRATES WHY THIS CAN BECOME POWERFUL

Imagine:

software.entity.yml

```
include:
- entity.entity

field:
  status:
    type: enum
    values:
- installed
    - unhealthy
    - outdated
    - unavailable

    executable:
    type: path

    files:
    type: path[]

    dirs:
    type: path[]

    dependencies:
    type: entity[]
```

Then:

tesseract.entity.yml

```
include:
- software.entity

entity:
  type: tesseract
```

Now Tesseract automatically participates in the `software` vocabulary.
And the concrete hook:
scripts/asc/contrib/asc/tesseract/...
provides the implementation.
That means ASC can eventually reason:

```
tesseract
  is software
  has executable
  has files
  has directories
  depends on ...
  provides recognize/text
```

without knowing anything specifically about Tesseract.
That is much more interesting than merely having a configuration file.

4. `*.able.yml` COULD BE EVEN MORE IMPORTANT THAN ENTITIES

I particularly like your contract idea.
Your current root entity already anticipates a capability/contract vocabulary, including `contract`, `required`, `optional`, and validation. (GitHub)
I would think of these as two different axes:
ENTITY
- "What is this?"
ABILITY / CONTRACT
- "What can this participate in?"
For example:
software.entity.yml
describes:
software
while:
ocr.able.yml
could describe:
something capable of OCR
Then:

```
tesseract.entity.yml
       ├── include software.entity
       └── include ocr.able
```

And another implementation:

```
easyocr.entity.yml
      ├── include software.entity
      └── include ocr.able
```

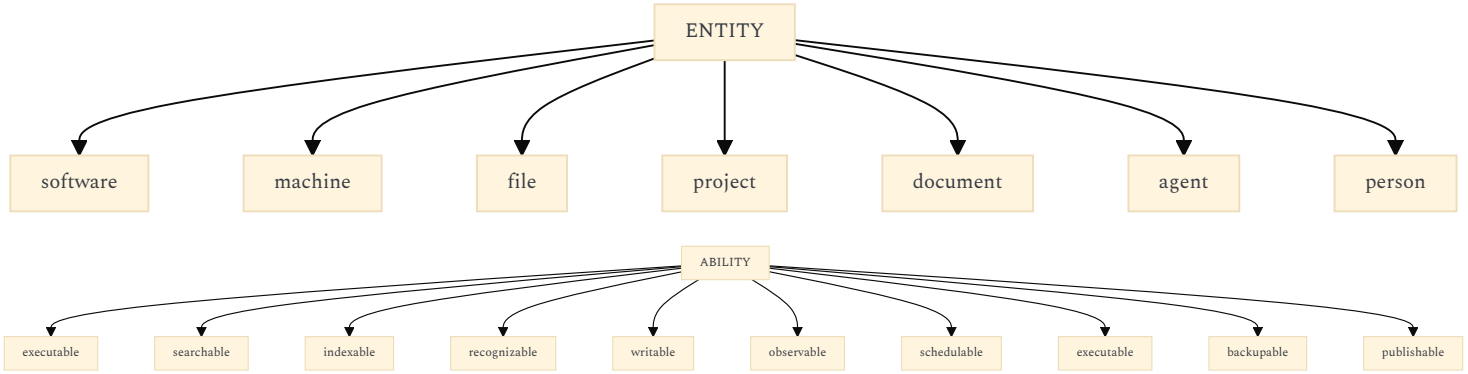
Now the UI doesn't need:

- if tesseract
- else if easyocr

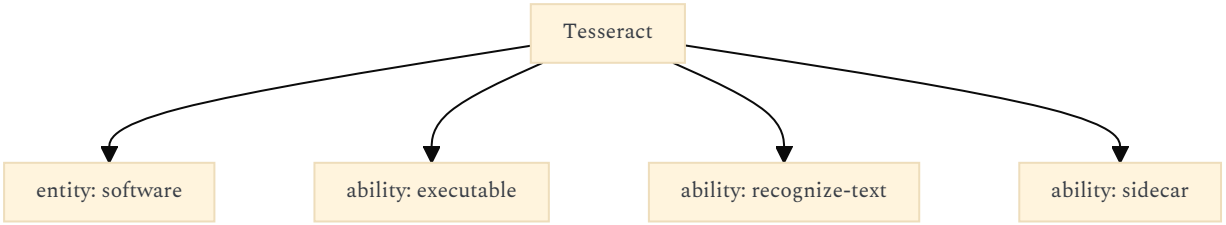
It can ask:
"what entities satisfy OCR?"
This is exactly the kind of abstraction that could later become extremely useful for agents.

5. IN FACT, I'D DISTINGUISH "ENTITY" AND "CAPABILITY" VERY SHARPLY

You could eventually have:



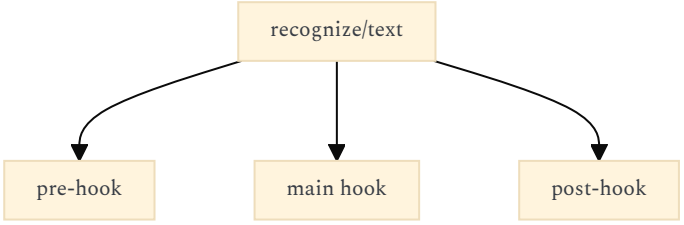
Then:



This gives your DSL a potentially very expressive vocabulary without making the DSL itself complicated.

6. THE OCR EXAMPLE IS AN EXCELLENT ARCHITECTURE

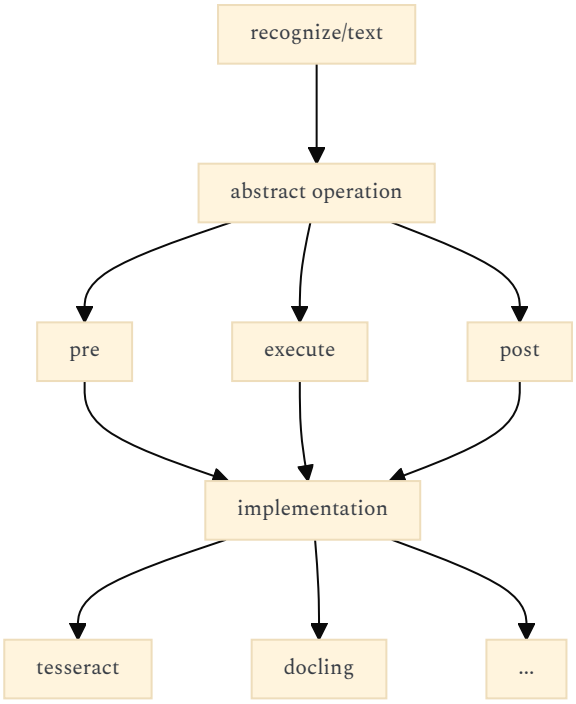
Your current abstract script essentially does:



and selects variants such as:

- STACK_VERSION
- HOST_OS
- PROVISION_USING

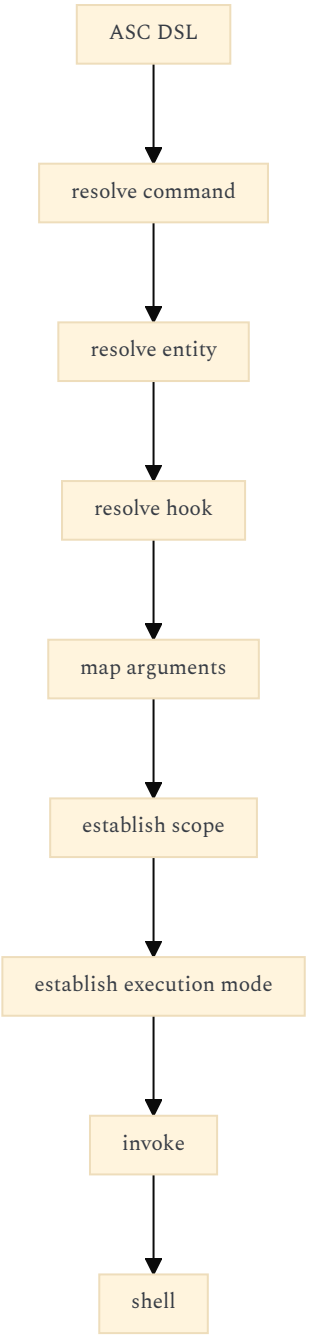
(GitHub)
That's already very close to a tiny execution protocol.
I'd preserve this.
Conceptually:



The abstract entry point doesn't care which implementation wins.
That's exactly the sort of swappability you want.

7. THEN YOUR DSL BECOMES A *SELECTOR/COMPOSER*, NOT ANOTHER SHELL

This distinction is crucial.
You gave:
test-in(foo,[bar,baz])
with equivalent execution through:
make test-in foo "'bar baz'"
or:
asc/utils/test/in.sh foo 'bar baz'
That's good.
But I would conceptualize:
test-in(foo,[bar,baz])
as an **ASC invocation expression**.
Not as an alternative shell syntax.
The shell is still the bottom layer.
So:



This keeps the DSL small.

8. YOUR `wrap` / `nest` DISTINCTION IS WHERE THINGS GET GENUINELY INTERESTING

I think you're touching an important distinction.
I'd model at least these three execution concepts:

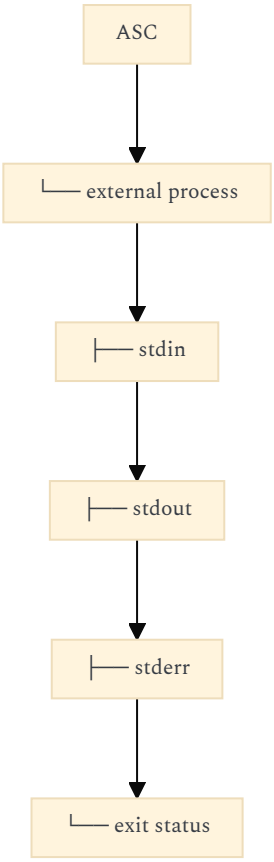
- invoke
- wrap
- nest

`invoke`

Run an operation.
ocr(file)

`wrap`

Treat an external program as an ASC operation.
wrap(tesseract,file)
Conceptually:



`nest`

Compose ASC operations inside another ASC execution context.

```
nest(
  recognize(file),
  index(result),
  relate(result)
)
```

That gives you a distinction between:
external execution
and:
ASC execution composition
I would preserve that distinction.

9. PARALLELISM SHOULD PROBABLY BE A PROPERTY OF COMPOSITION

Rather than creating a complicated execution language, you could eventually have something conceptually like:

```
batch(
  a(),
  b(),
  c()
)
```

versus:

```
sequence(
  a(),
  b(),
  c()
)
```

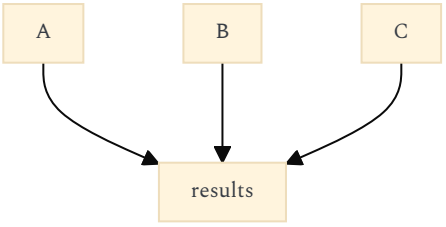
and perhaps:

```
pipe(
  a(),
  b(),
  c()
)
```

That gives you three fundamentally different semantics:
sequence

```
A → B → C
```

parallel



pipe

```
A → B → C
```

You don't need a giant workflow engine.
You need a tiny compositional vocabulary.

10. THIS IS WHERE AGENTS COULD EVENTUALLY BECOME NATIVE TO ASC

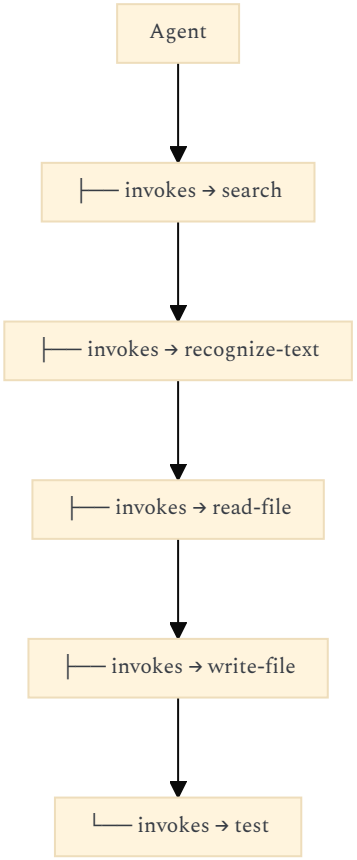
This is perhaps the biggest consequence of what you're describing.
An agent could eventually be represented as an ASC entity:
include:

- entity.entity
- software.able
- executable.able
- observable.able

with capabilities such as:

- agent/run
- agent/stop
- agent/status
- agent/log
- agent/ask
- agent/approve

And then the second brain could display an agent's execution using the same underlying abstraction as any other ASC operation.

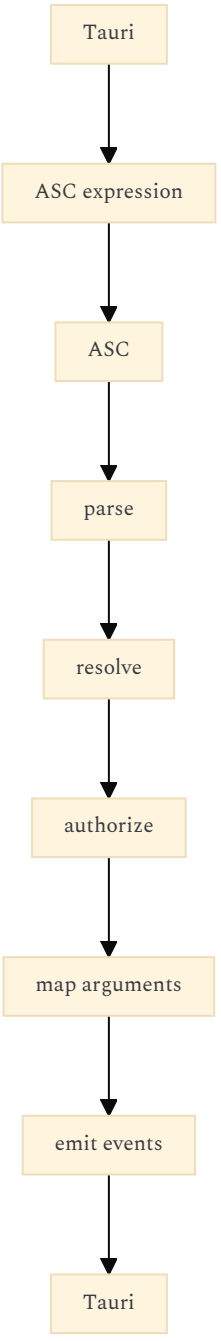


The UI isn't special-casing AI.
It is visualizing **ASC execution**.

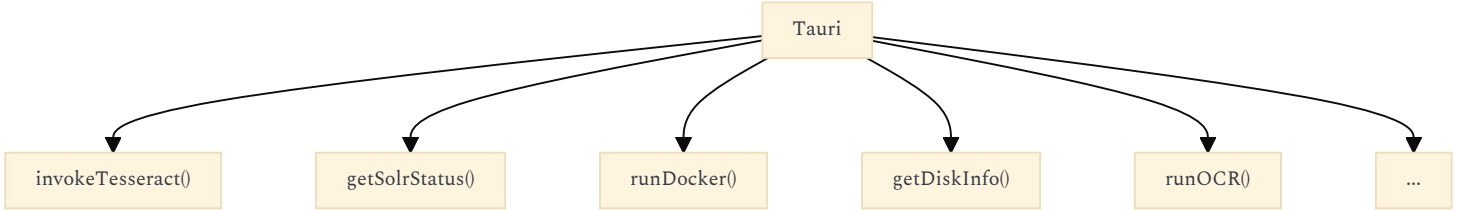
That is a much stronger foundation.

11. THIS ALSO GIVES YOU AN ELEGANT ANSWER TO THE TAURI/ASC BOUNDARY

I would now make the Tauri application communicate with **ASC expressions and events**, rather than inventing an application-specific API.
For example, the UI could conceptually request:
recognize-text(path/to/file.jpg)
or:
test-in(foo,[bar,baz])
or:
software/status(tesseract)
ASC resolves them.
The UI receives structured results/events.



This is far cleaner than:



12. THE FILENAME-SAFE ARGUMENT NAMING PROBLEM IS REAL

I would be conservative here.
You essentially have several namespaces:

- filesystem
- ASC entity
- ASC operation
- argument
- property
- contract
- hook
- scope

and they eventually need to coexist in filenames.
I'd strongly consider adopting a restricted identifier grammar early.
Something along the lines of:

```
[A-Za-z0-9][A-Za-z0-9._-]*
```

with a separate convention for hierarchy:

- software/tesseract
- recognize/text
- agent/run
- project/status

But avoid putting arbitrary shell syntax into identifiers.
For example:
foo/bar
can represent namespace/path semantics,
while:
foo-bar
remains one identifier.
Then argument names can remain:

- input-file
- output-dir
- host-name
- agent-id

rather than:

- input file
- input/file
- input$file

The critical rule is:

> **An ASC identifier should always be safely representable as a filename component without escaping.**

That gives you a very useful invariant because ASC already maps concepts onto actual filesystem paths.

13. I WOULD NOT MAKE ARGUMENT MAPPING IMPLICIT EVERYWHERE

This is probably the biggest unresolved part of the design.
You have:

```
test-in(foo,[bar,baz])
```

but eventually you need to distinguish:
`foo`
from:
`[bar,baz]`
and determine whether that means:

- argv
- stdin
- environment
- file
- glob
- entity reference
- property reference
- literal
- expression

I'd establish a very small type vocabulary.
Something conceptually like:

- literal
- path
- entity
- field
- list
- expression
- stdin
- stdout

Then:
`recognize-text(file)`
can resolve:
`file → path`
while:
`software/status(tesseract)`
resolves:
`tesseract → entity`
and:
`test-in(foo,[bar,baz])`
could resolve:

```
foo       → literal
[bar,baz] → list
```

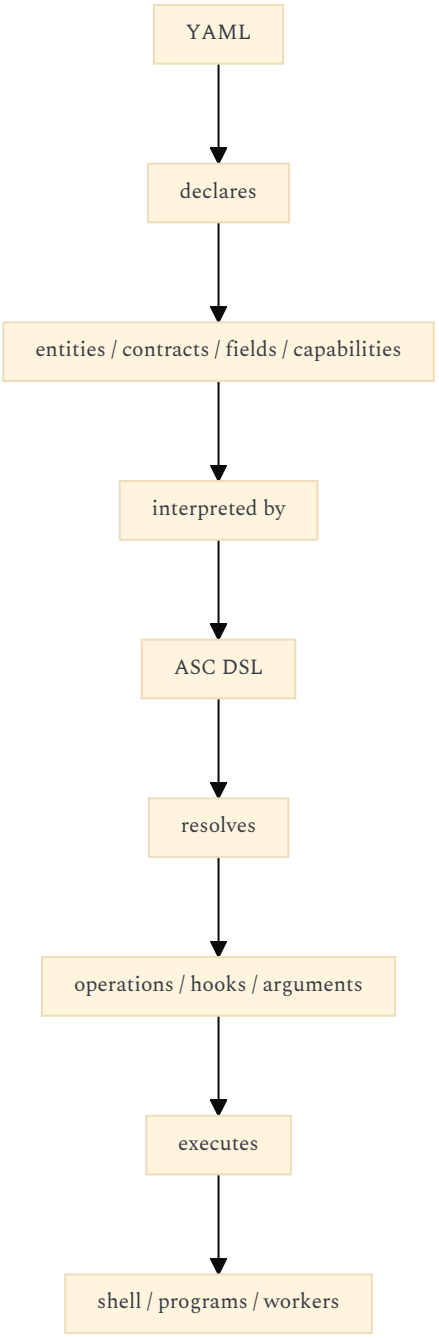
This prevents the DSL from eventually becoming ambiguous shell interpolation.

14. THERE IS A SECOND DISTINCTION WORTH INTRODUCING: VALUE VS REFERENCE

For a knowledge-oriented system this becomes essential.
Compare:
foo
with:
@foo
or some equivalent notation.
The first could mean:
literal string "foo"
while the second means:
reference to entity foo
You don't necessarily need `@`; the syntax is only illustrative.
But the semantic distinction is important.
Otherwise:
software/status(tesseract)
becomes ambiguous:
"tesseract"
or:
the entity called tesseract
That distinction will become critical when agents start composing operations.

15. YOUR YAML + DSL COMBINATION IS POTENTIALLY A TINY REFLECTIVE SYSTEM

This is where I think your design becomes much more interesting than I initially understood.
You could have:



And then ASC can potentially introspect itself:

- "What is tesseract?"
- "What can tesseract do?"
- "What does recognize-text accept?"
- "Which implementations satisfy OCR?"
- "What dependencies does this software have?"
- "Which machine provides this capability?"

The same declarations that tell ASC how to execute something can tell the Tauri UI how to **represent it**.
That is a major opportunity.

16. THIS IS EXACTLY WHERE `projet-complexe` CAN EXPLOIT ASC WITHOUT BECOMING COUPLED TO ITS IMPLEMENTATION

Imagine the UI receives an entity description:

- tesseract
- entity: software
- capabilities:
  - executable
  - recognize-text
- fields:
  - status
  - executable

    - files
    - dirs
    - dependencies

It can automatically produce a visual representation:

```
Tesseract
──────────────────────────
status       installed
executable   /usr/bin/tesseract
capabilities
  recognize-text
dependencies
  ...
files
  ...
directories
  ...
```

The UI doesn't need a hard-coded Tesseract component.
That's potentially one of the most powerful consequences of your YAML approach:

> **ASC declarations can become the metadata from which the second brain constructs its visual model of the computational world.**

17. AND THE SAME MECHANISM CAN WORK FOR KNOWLEDGE OBJECTS

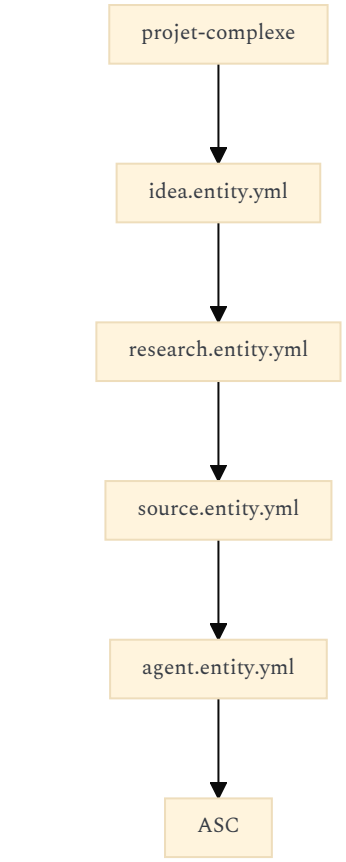
This is where the two repositories converge elegantly.
ASC could define:

- entity
- software
- machine
- file
- process
- service

while `projet-complexe` could extend the vocabulary:

- project
- idea
- research
- source
- concept
- note
- task
- agent
- publication

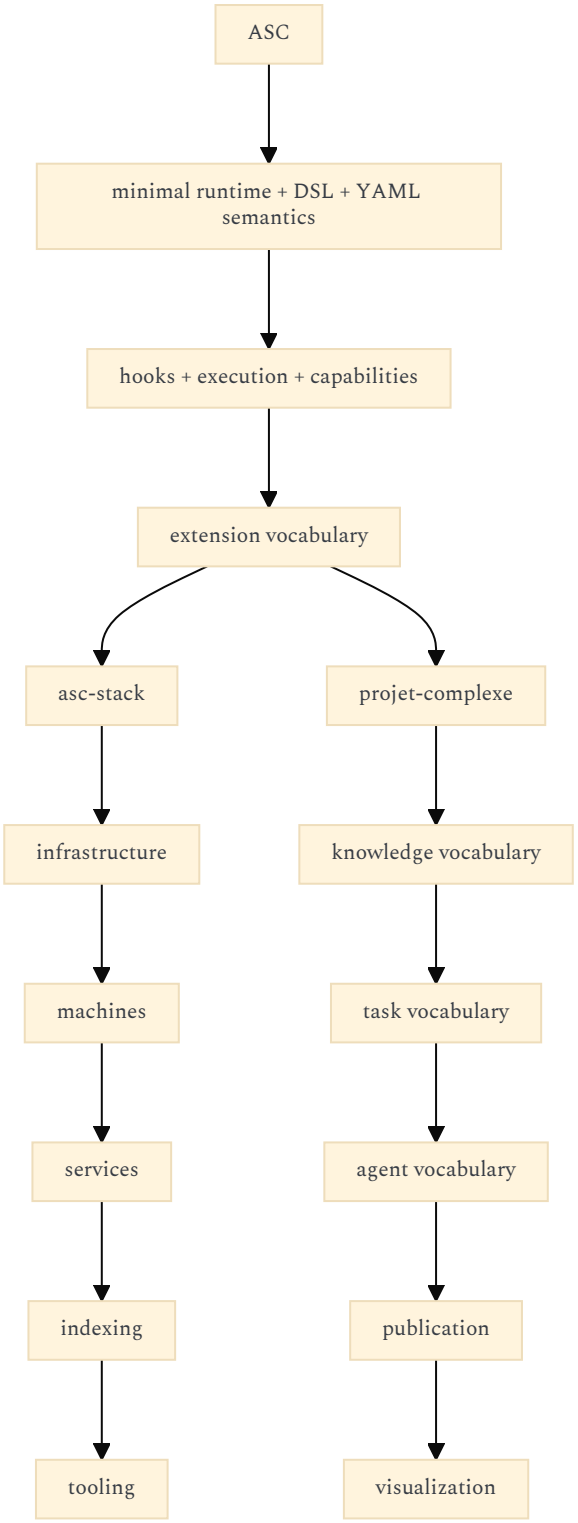
They could all use the same declaration mechanism.
Not because `projet-complexe` becomes part of ASC, but because ASC provides the **minimal reflective substrate**.
For example:



This makes the repository boundary much less problematic.
ASC provides the language.
`projet-complexe` provides a vocabulary built with that language.

18. THAT SUGGESTS AN EVEN BETTER REPOSITORY RELATIONSHIP

I would now conceptualize your repositories as:



This is better than:

```
ASC
 └── second brain
```

because ASC remains generic.
And better than:

```
projet-complexe
 └── its own ASC fork
```

because you don't duplicate the execution substrate.

19. ONE THING I WOULD *NOT* DO YET

Don't try to specify the entire DSL.
Your current `entity.entity.yml` is already signalling a very broad vocabulary:

- is
- access

- include
- field
- triple
- link
- synonym
- override
- hook
- wrap
- nest
- sidecar
- entity
- taxonomy
- cognition
- contract
- required
- optional
- ...

(GitHub)
That is a lot.
The danger is designing the language before you have enough real operations to constrain it.
I would instead implement perhaps:

- include
- entity
- field
- hook
- contract

first.
Then:

- wrap
- nest

when actual composition requires them.
Then:
parallel / sequence / pipe
when actual workflows require them.
Then let the vocabulary grow from real use.
Your `yml.inc.sh` already represents a substantial custom YAML-processing layer, so keeping the supported YAML subset deliberately small is particularly valuable for maintainability. (GitHub)

20. THE RESULTING SYSTEM HAS A VERY NICE RECURSIVE PROPERTY

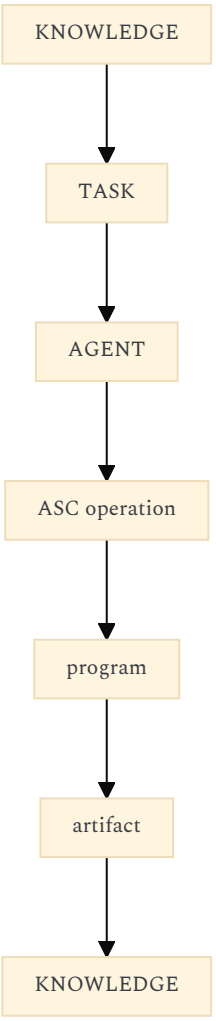
This may be the most interesting part.
You want ASC to represent:

> anything that can be done in the shell.

Then you want `projet-complexe` to represent:

> anything you want to understand and organize.

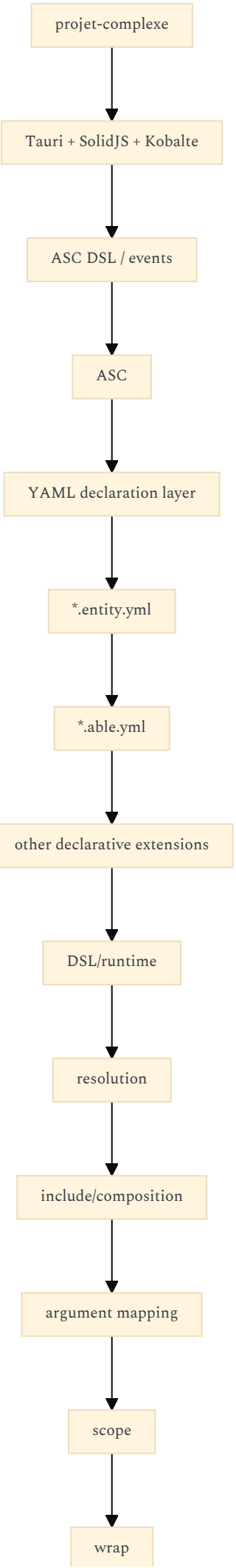
If ASC operations are themselves entities/capabilities, then the second brain can represent **the operations that manipulate the second brain**.
For example:

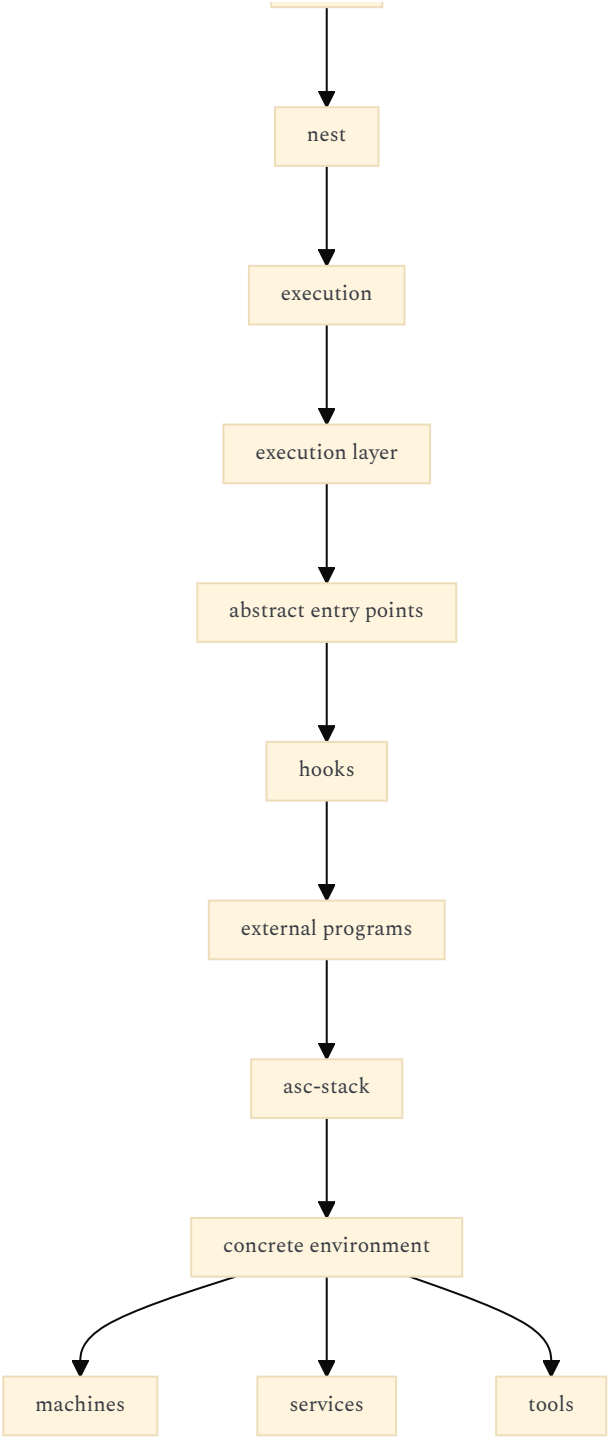


The system becomes reflexive.
That is very different from a normal CRUD application.

21. MY CURRENT ARCHITECTURAL RECOMMENDATION

I would now settle on this:


projet-complexe
Tauri + SolidJS + Kobalte
ASC DSL / events
ASC
YAML declaration layer
*.entity.yml
*.able.yml
other declarative extensions
DSL/runtime
resolution
include/composition
argument mapping
scope
wrap



And the crucial conceptual equations would be:

```
ENTITY
    = description of something

ABILITY / CONTRACT
    = description of what something may provide/require

DSL EXPRESSION
    = request to operate on something

HOOK
    = concrete implementation of an operation

WRAP
    = make an external executable participate in ASC

NEST
    = compose ASC execution contexts

ASC
    = resolver + executor + declarative substrate
```

That is enough to make your "protocol" live **inside ASC**, without creating a protocol repository.
The real thing to design carefully now is not another repository. It is the **semantic boundary between YAML declarations, DSL expressions, and executable hooks**. If that boundary remains tiny and orthogonal, `projet-complexe` can become a very sophisticated second brain and agent controller without

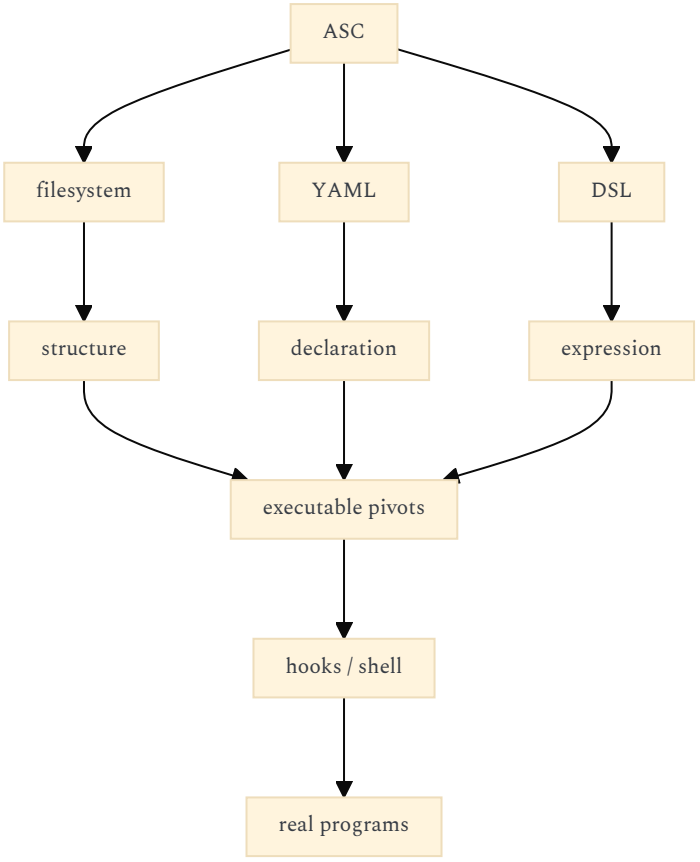
turning ASC into a second-brain-specific framework.

FOLLOW-UP 3 : YOU COULD HAVE READ MY README FIRST (BUT WE'LL MANAGE)

I already have written some clarifications regarding some of the suggestions and / or interrogations you made, here's what I had in mind : (see attached) can you go through it and revise your answer where appropriate ?
https://chatgpt.com/s/t_6a7d04b86f94819196cde5e0cf07bd34
The attached notes substantially change my previous answer in a useful way. I was still treating ASC too much like a conventional **execution framework with a declarative metadata layer**. Your notes describe something more radical and, I think, more coherent:

> **ASC is becoming a filesystem-oriented, declarative/compositional language for representing and executing computational things, where YAML, paths, sidecars, entry points, hooks, DSL expressions and shell programs are different representations of the same underlying structure.**

That means several of my previous recommendations should be withdrawn or reframed.
The strongest architectural consequence is this:



And `projet-complexe` should indeed sit **above this**, but not as a conventional "client of an ASC API". It should become a **visual/semantic projection of the ASC universe**, with its own task/knowledge vocabulary.
Confidence: 0.97.

1. THE MOST IMPORTANT CORRECTION: ASC IS NOT PRIMARILY AN API

My previous answer said things like:

> "Tauri should communicate with ASC expressions and events."

That remains directionally correct, but the conceptual model was too API-centric.
Your notes make clear that the **filesystem itself is part of ASC's language**.
For example:

- $subject/$action
- $subject/$object/$action

are not merely implementation paths. They carry semantic information.
Likewise:

- [entity].entity.yml
- [able].able.yml
- [action].sh
- [hook].{file_ext}
- {subject}.inc.sh

are all syntactic forms in a larger naming grammar.
So I would now describe ASC as:

```
ASC =
    filesystem grammar
  + declarative grammar
  + executable grammar
  + composition grammar
  + resolution rules
```

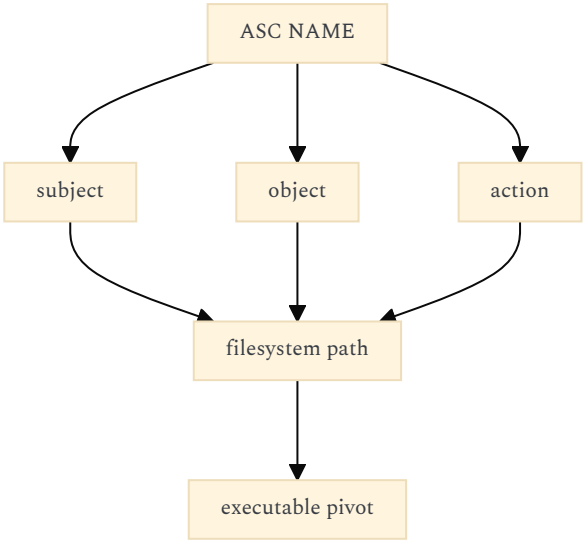
rather than:
ASC = runtime + protocol
This makes the absence of a separate `asc-protocol` project even more obvious.

2. `$subject / $object / $action` IS NOT JUST A PATH CONVENTION

Your notes reveal something I missed.
You are explicitly considering multiple possible filesystem representations:
$subject / $action
and:
$subject / $object / $action
with ASC remaining deliberately agnostic about whether an entity chooses to implement that extra level.
That means `$subject`, `$object`, and `$action` are better understood as **semantic coordinates** which may be projected into different filesystem structures.
The distinction:

- remote-host--foobar
- remote-host--reverse-proxy--state

also makes this explicit: `--` means a relation rather than an entry point.
So I would now model ASC's namespace roughly as:



The filesystem is therefore not merely storage. It is **part of the semantic addressing system**.
That is a much stronger idea.

3. YOUR "ENTRY POINTS ARE FIXED PIVOTS" PRINCIPLE SHOULD BE CENTRAL

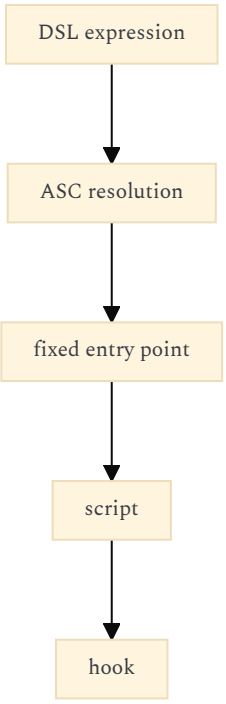
This is probably the single most important sentence in the notes:

> **entry points are fixed pivots, and are sidecar.able as pre-compiled commands (cmd).**

That changes how I would think about the DSL.
I previously described the DSL as a command language that resolves to executable operations.
I would now say:

> **The DSL is a compact notation for addressing/composing ASC's fixed pivots.**

For example:
`test-in(a1,[slug(a-1,-),slug(a-1,_)])`
doesn't necessarily represent an abstract function invocation in the programming-language sense.
It represents something closer to:



And your intention to make `make` understand DSL as a fallback reinforces this: DSL becomes another way of addressing the same ASC machinery, rather than a second execution system.

4. THIS ALSO CHANGES MY VIEW OF `wrap` AND `nest`

I previously proposed:

- invoke
- wrap
- nest

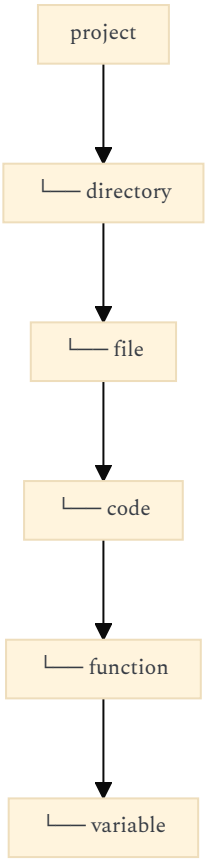
as three execution primitives.
Your notes suggest something subtler.
`nest.able` is not merely "nested execution".
You explicitly connect it to:

> `nest.able = zoom.able`

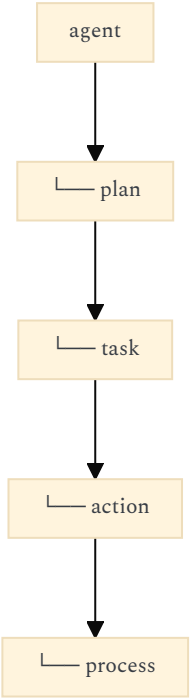
and eventually to graphical navigation through trees and fractal structures.
That is much more interesting.
I would therefore **not define `nest` primarily as an execution primitive**.
Instead:

- nestable
- =
  something that can contain / expose a subordinate ASC structure

Execution can happen inside it, but nesting is fundamentally a **structural property**.
For example:



and:



could all share the same structural mechanism.
That makes your "fractal navigation" idea considerably more compelling.

5. `*.able.yml` IS NOT REALLY AN INTERFACE IN THE OO SENSE

My previous answer came close to treating `*.able.yml` as a conventional capability/interface system.
Your notes point toward something more general.
You describe:

- field = stored instance value
- prop = YAML constant shared by entities

with fields being editable/stored and props being inherited/composed YAML constants.
Then:
able
can define a reusable structural/behavioral contract.

I would therefore avoid imposing the OO analogy too strongly.
Instead:

```
.entity.yml
    = what this kind of thing declares

.able.yml
    = reusable declaration/constraint/structure

.field
    = instance state

.prop
    = declaration-level constant

.include
    = composition
```

That is closer to **structural composition** than inheritance.
Your own "include = merge" idea therefore fits better than my earlier inheritance terminology.

6. I WOULD KEEP "INCLUDE = COMPOSITION", NOT "INHERITANCE"

This is one place where your notes resolve the ambiguity.
You explicitly say:

> hard rule: all include of entities must be namespaced

and give:
asc.contract
rather than an unqualified `contract`.
That suggests `include` is fundamentally a **namespace-resolved composition mechanism**.
So I would now formulate it:
include(A, B)
means roughly:

```
resolve B
    +
merge B's declaration
    +
resolve conflicts according to ASC rules
```

rather than:
class A extends B
This is a better fit for your system.

7. THE "GENERICITY SCALE" IS MORE IMPORTANT THAN I PREVIOUSLY REALIZED

Your six-level scale is extremely useful:

1. primordial
2. primitive
3. ASC core extensions
4. ASC contrib extensions
5. third-party contrib extensions
6. project-specific implementations

This gives us a much better answer to the question:

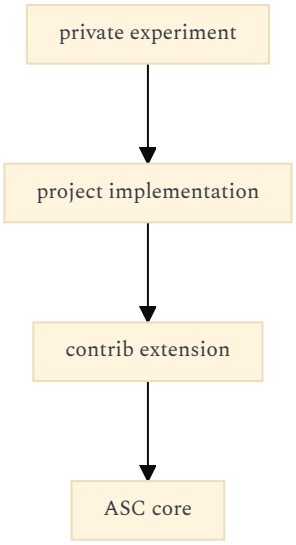
> Where does `projet-complexe` belong?

It should **not modify the meaning of ASC primitives**.
Instead it should live predominantly around levels 5–6:

And your `asc-stack` would mostly be:

- level 6
- project-specific environment

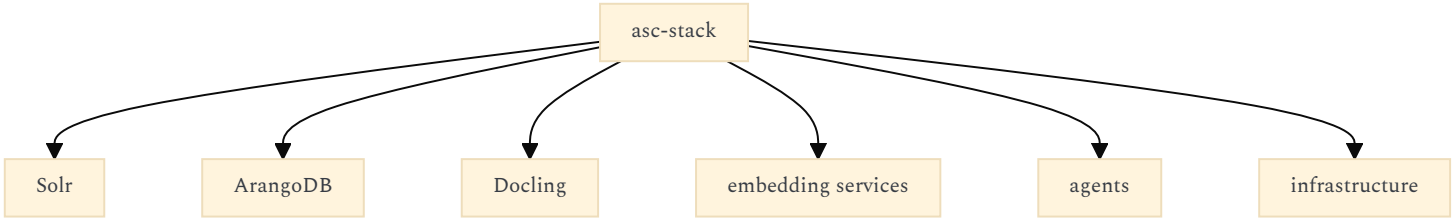
with perhaps some reusable contributions eventually migrating upward.
That gives you a natural evolutionary mechanism:



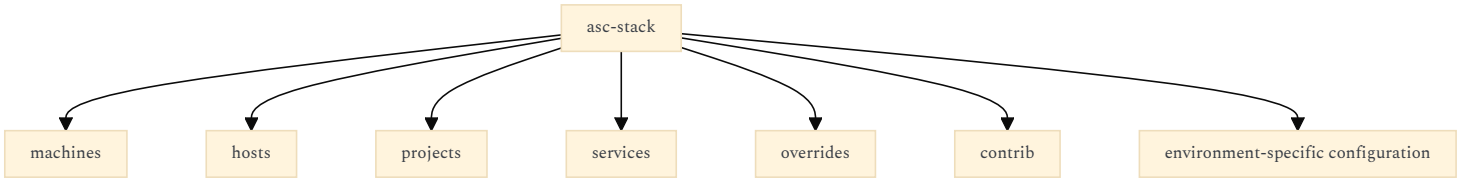
without requiring you to decide upfront what belongs where.

8. THIS CHANGES MY RECOMMENDATION FOR `asc-stack`

I still prefer your **separate repository** architecture.
But I would now make `asc-stack` considerably thinner than I previously suggested.
I previously imagined it as something like:



Your notes suggest that much of this should actually become **ASC entities, sidecars, commands and extensions**.
So:
asc-stack
should primarily be a **composition/configuration of your personal ASC environment**.
For example:



while reusable semantics go back into ASC.
This is more like:

- ASC = language/runtime
- asc-stack = my deployment/environment
- projet-complexe = visual + semantic application

9. YOUR SIDECAR CONCEPT IS MUCH MORE FUNDAMENTAL THAN I UNDERSTOOD

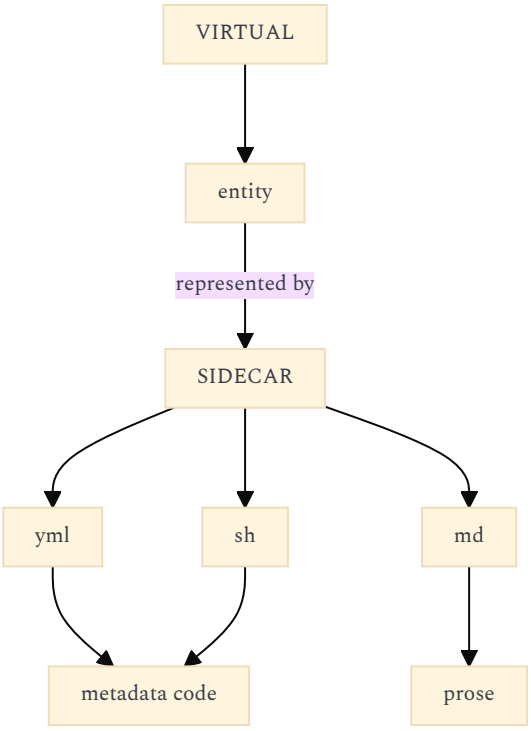
This is probably the biggest thing I would add to my previous answer.
You write:

> Entity = represents something virtual
> Yml = concrete (actual) file sidecar

and:

> Any script can be concrete (actual) file sidecar of any action.

This gives you an extremely elegant duality:



The same concept can therefore have different concrete representations.
That is exactly what your second brain will need.
A research entity might have:

- research.entity.yml
- research.md
- sources/
- notes/
- data/

A project might have:

- project.entity.yml
- README.md
- src/
- tests/
- compose.yml

An agent might have:

- agent.entity.yml
- prompt.tpl
- config.yml
- logs/
- plans/

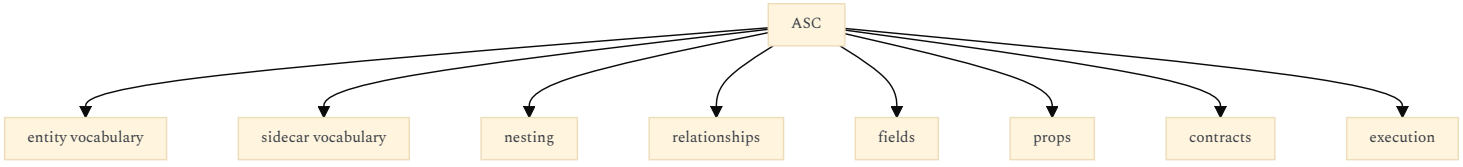
The entity is the **virtual conceptual object**.
The sidecars are its **concrete manifestations**.
That is a much stronger foundation for your second brain than a conventional database model.

10. CONSEQUENTLY, I WOULD REVISE MY EARLIER "KNOWLEDGE LAYER" RECOMMENDATION

I previously suggested something like:

```
ASC
└── Knowledge layer
```

I would no longer make that a separate architectural layer.
Instead:



Then `projet-complexe` **defines a knowledge vocabulary using ASC**.

For example:

```
projet-complexe/
  extensions/
    knowledge/
      source/
      concept/
      research/
      note/
    task/
      task/
      project/
      change/
    agent/
      agent/
      plan/
      run/
```

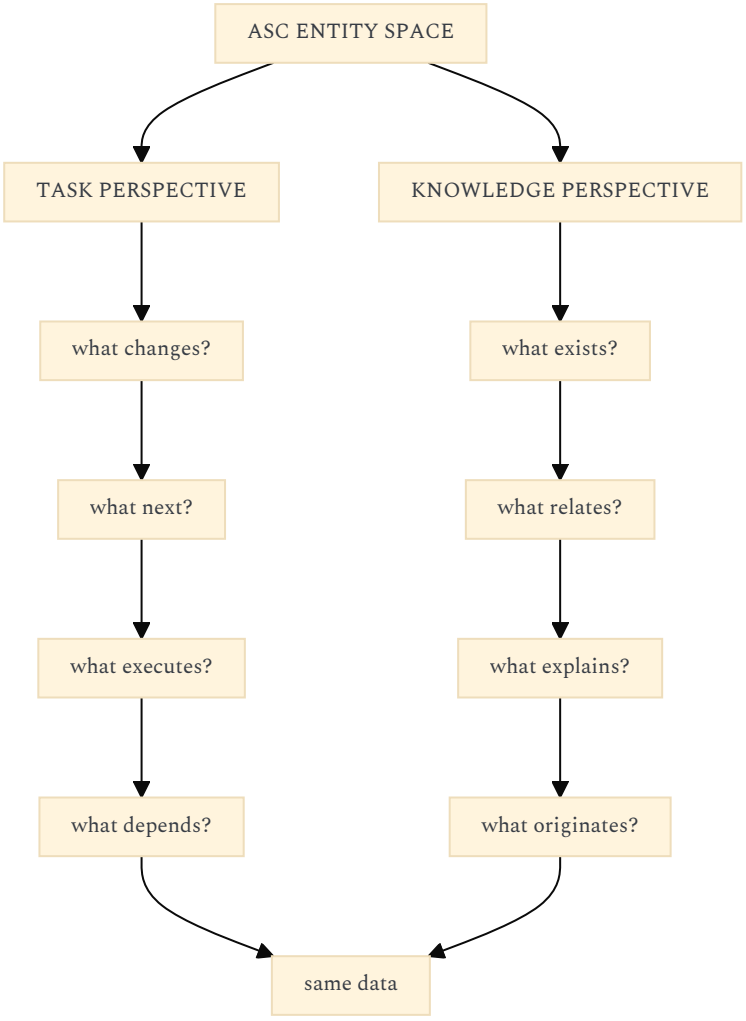
That is cleaner.

11. THE TASK/KNOWLEDGE SPLIT BECOMES EVEN MORE INTERESTING

Your note:

> **task-oriented VS knowledge-oriented: mutual killswitch to implement**

is much more important than it looked in our previous discussion.
I would preserve it not merely as two UI modes but as two **orthogonal perspectives over the same entity graph**.
Something like:



The "killswitch" could therefore mean:

```
task mode
→ hide knowledge-centric affordances

knowledge mode
→ hide execution-centric affordances
```

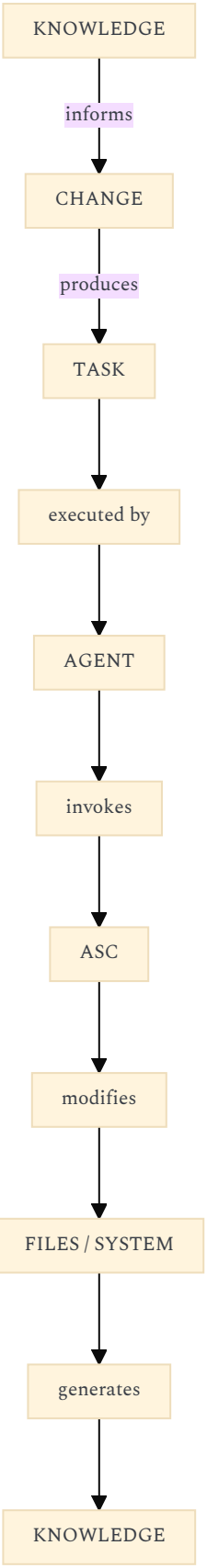
without maintaining two separate data models.
That is very compatible with the second-brain ambition.

12. THE "CHANGE" ENTITY MAY BE THE BRIDGE BETWEEN THE TWO

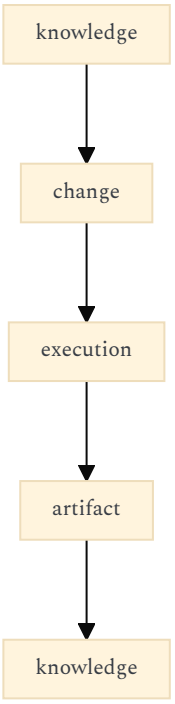
Your workflow notes say:

> Workflow: centered around change; changelogs are change sidecars; changes must be entities.

This is potentially the missing central concept.
Consider:



That creates a cycle:



This fits your "everything is a reformulation problem" idea extraordinarily well.
A change isn't simply a Git commit.
It is an **ASC entity representing an intended/realized transformation**, potentially with Git providing the underlying mechanics.

13. I WOULD THEREFORE NOT MAKE THE SECOND BRAIN PRIMARILY A DATABASE

This is another consequence of your notes.
Your relations:

- subject--object
- subject--predicate--object

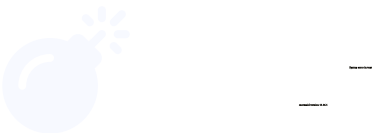
are explicitly described as useful notation but with mapping to complex memory stores out of scope.
That reinforces an architectural principle:

> **Don't prematurely turn ASC into a graph database.**

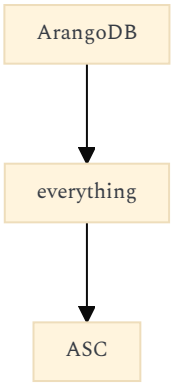
ArangoDB/Solr/etc. can eventually be indexing/projection infrastructure.
But ASC's primary representation can remain:

```
filesystem
   +
  YAML
   +
sidecars
   +
relations
```

The database becomes an **index/projection**, not the canonical representation.
For your second brain, this is particularly attractive.

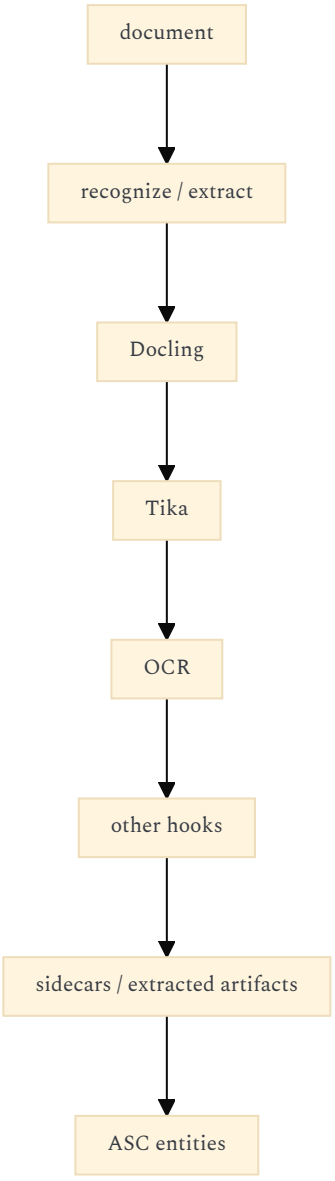


rather than:



14. THIS ALSO CLARIFIES WHERE DOCLING BELONGS

I previously put Docling in the "knowledge layer".
I'd now put it much lower:



So Docling isn't a semantic dependency of `projet-complexe`.
It is one implementation of an ASC capability.
That makes it swappable.

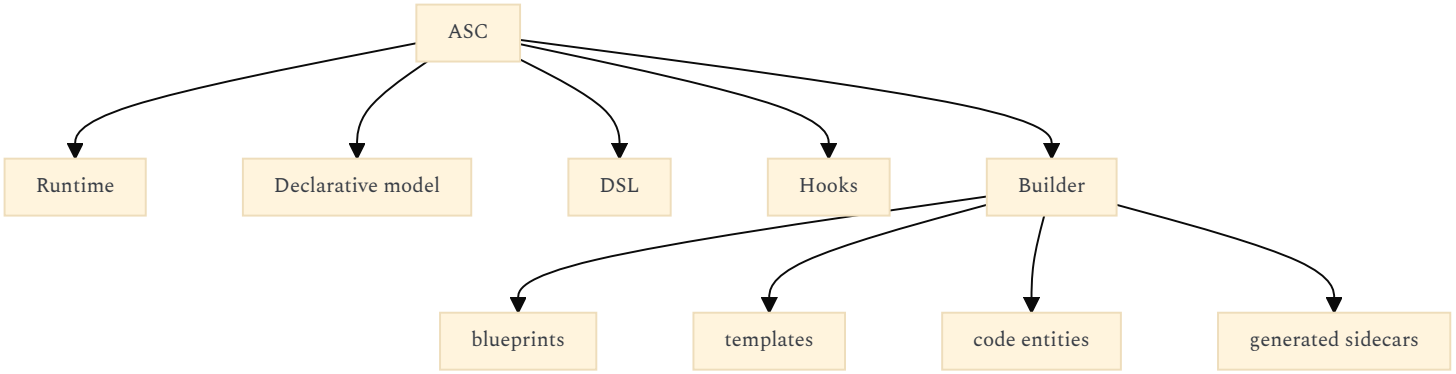
15. YOUR BUILDER IS ALSO MORE IMPORTANT THAN I INITIALLY UNDERSTOOD

The Builder isn't just a code generator.
Your notes explicitly connect:

- blueprint
- atomic
- nestable

- usable
- templates
- files
- dirs
- vars
- functions
- ASC instances

and even propose using a single `atomic.able` representation.
That suggests Builder is effectively ASC's **metaprogramming layer**.
So I'd now conceptualize:



The Builder can therefore produce ASC structures that ASC itself can subsequently consume.
That's a reflective loop.

16. THE TEMPLATE SYSTEM IS ESSENTIALLY A SECOND DSL

Your template examples show:

```
{{ slot }}
<asc-if ...>
<asc-for ...>
{{ entity-preview(...) }}
```

This means you actually have at least two syntactic languages emerging:
ASC DSL
for **execution/addressing/composition**,
and:
ASC template DSL
for **generation/projection**.
I would resist making them identical.
But they should probably share the same resolver.
For example:
should ideally invoke the same DSL evaluator as:
make 'test-in(...)'
rather than creating a separate semantic system.
That would be a major architectural invariant:

> **There should be one ASC semantic evaluator, regardless of whether the expression originated from a shell, a template, the Builder, or Tauri.**

17. THIS MAKES THE TAURI APPLICATION MUCH MORE INTERESTING

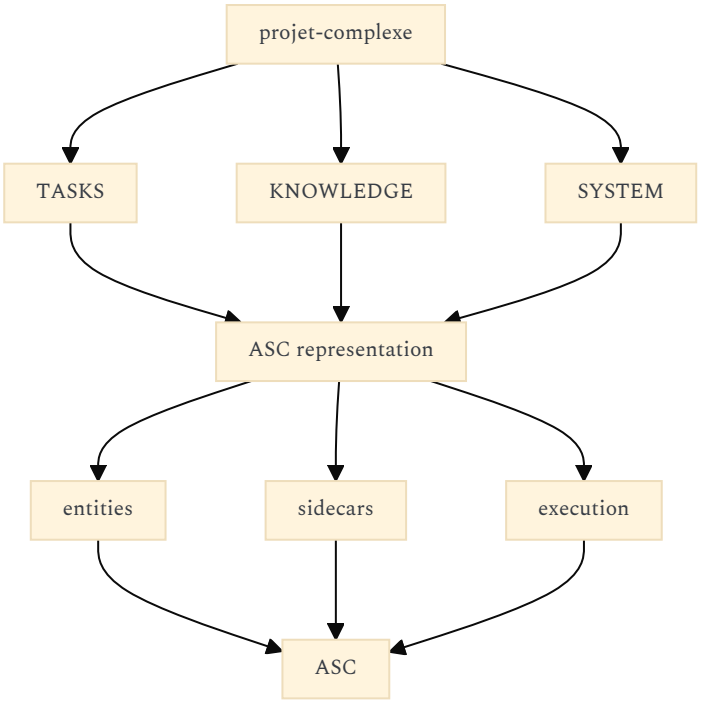
I would now describe `projet-complexe` less as:

> Tauri application controlling ASC

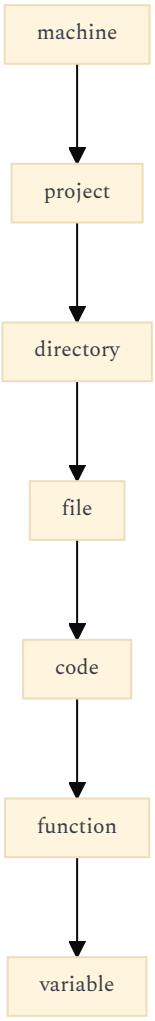
and more as:

> **a graphical projection engine for ASC entities, structures, relations, changes and executions.**

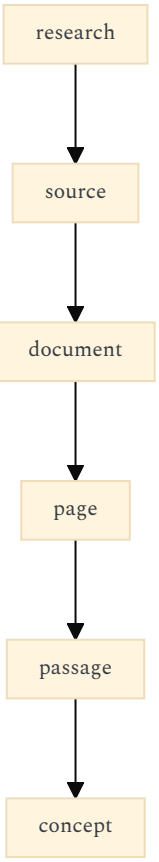
Something like:



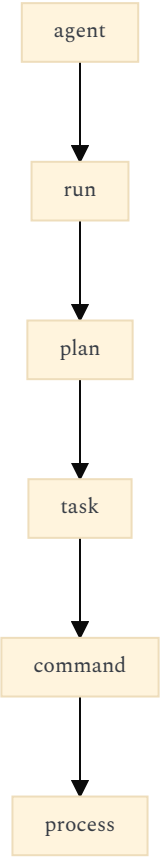
The UI can then zoom through the same nested structures:



or:



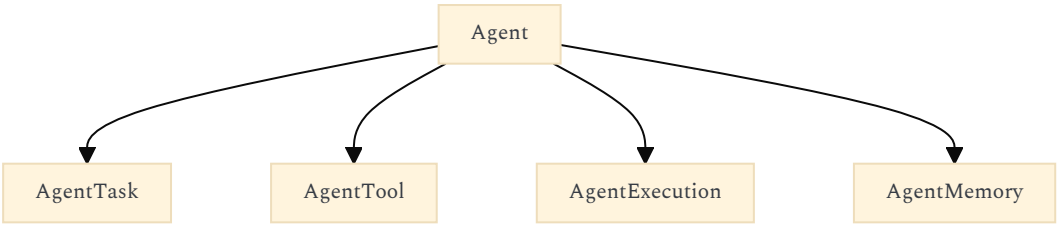
or:



This is exactly where your `nest.able = zoom.able` idea becomes potentially foundational rather than cosmetic.

18. THE AGENT CONTROLLER SHOULD THEREFORE NOT HAVE ITS OWN OBJECT MODEL

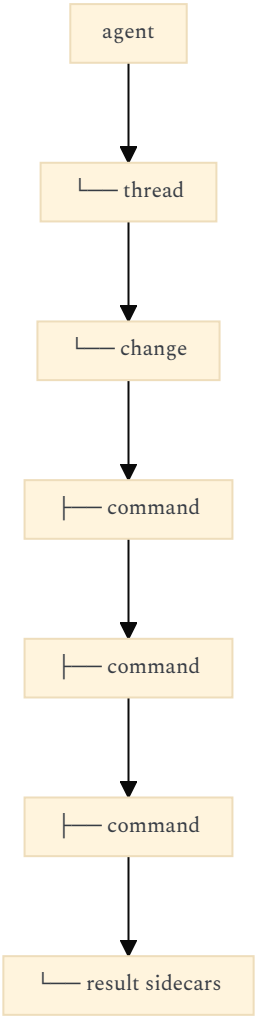
This is another change I would make to my previous recommendation.
Don't build:



as a completely separate application model unless experience forces it.
Instead, let agents participate in the existing ASC vocabulary:

- agent.entity
- agent.able
- process/thread
- task
- change
- sidecar
- command
- hook

Then an agent run becomes something like:



This is much closer to your stated ambition of making agents visible as things happening **inside the computational environment**.

19. YOUR `thread` DECISION NOW MAKES SENSE

You initially questioned `thread` vs `process`, then decided to retain `thread` because ASC threads represent **ASC-managed process-like pivots**, distinct from arbitrary shell processes.
I think that is coherent.
Don't redefine it according to operating-system terminology.
ASC's ontology is allowed to have:
ASC thread
meaning:

> a process/execution context managed and represented by ASC.

Then an actual Linux process can be a concrete implementation detail.
This is another example of why ASC shouldn't simply mirror POSIX.

20. I WOULD CHANGE ONE OF MY PREVIOUS RECOMMENDATIONS ABOUT ARGUMENT TYPING

I previously suggested introducing a type vocabulary such as:

- literal
- path
- entity
- field
- list
- expression

I still think the underlying problem is real, but your current DSL notation already suggests a much more deliberately **shell-oriented** solution:

- a
- a-1
- a-1s
- o-max-4
- bo-y

I would therefore **not introduce a large explicit type system yet**.
Instead, first stabilize:

- a
- a-N
- a-Ns
- o-name
- bo-name

and let entry points establish interpretation.
That keeps the DSL aligned with your stated goal:

> represent anything that can be done in the shell as a custom DSL.

The more type-rich semantics can emerge later through entity/field resolution.

21. THE ARGUMENT MAPPING IS ACTUALLY BEAUTIFULLY COMPATIBLE WITH YOUR FIXED-PIVOT MODEL

For example:
`test-in(a1,[slug(a-1,-),slug(a-1,_)])`
can compile conceptually to:

- $1
- "$1"-?

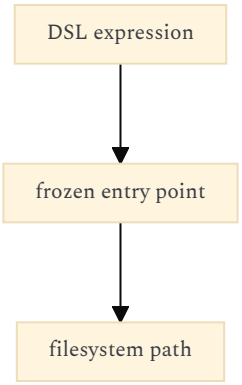
while:
o-max-4
maps to an option form.
The important point is that the DSL isn't trying to replace Bash's `$@`, `$1`, `$2`, etc.
It is **naming those positional/option slots declaratively**.
So I'd keep your proposed mapping:

```
a       → $@
a-1     → $1
a-N     → $N
a-1s    → shifted/rest arguments
o-X     → named option
bo-X    → boolean option
```

and make the DSL compiler/resolver responsible for preparing the shell scope.
Your own proposed `dsl()` implementation makes this explicit.

22. THE "FILENAME-SAFE" CONSTRAINT SHOULD BECOME A HARD INVARIANT

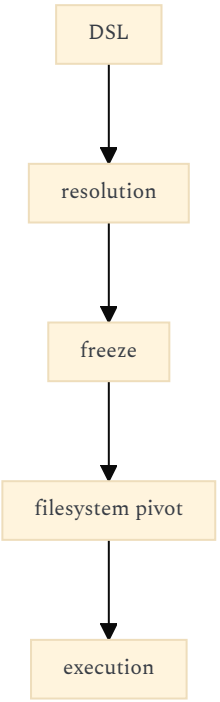
I strongly agree with this after reading the notes.
You have multiple reasons for it:



and you are considering paths such as:
`entity-field-val(type,a-1).dsl.hook`
or:
`entity-field-val(type,a-1).dsl.hook.yml`
So the filesystem isn't merely a cache.
It is part of the language's **compiled representation**.
That makes filename safety a semantic constraint rather than a convenience.

23. THIS ALSO MAKES "FROZEN DSL ENTRY POINTS" MAKE SENSE

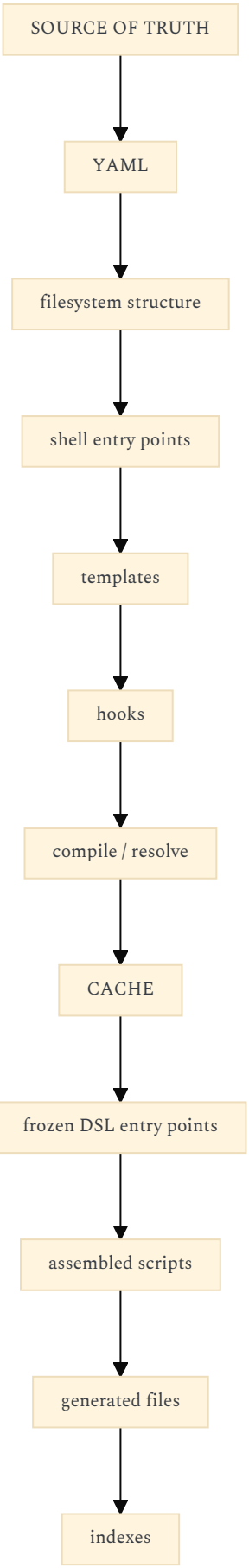
I was initially skeptical about having both DSL and filesystem entry points.
Your notes make the reason clearer.
You can have:
dynamic DSL
and potentially:
frozen DSL entry point
with a filename such as:
`entity-field-val(type,a-1).dsl.hook`
The latter becomes something like a compiled/cached addressable artifact.
Then:



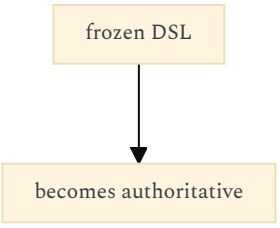
This fits extremely well with your Builder/cache ideas.
I would therefore keep this possibility.
The unresolved issue is collision handling and cache invalidation, which your notes explicitly identify.

24. I WOULD TREAT FROZEN DSL AS A CACHE, NOT AS A SECOND SOURCE OF TRUTH

This is important.
The hierarchy should be:


SOURCE OF TRUTH
YAML
filesystem structure
shell entry points
templates
hooks
compile / resolve
CACHE
frozen DSL entry points
assembled scripts
generated files
indexes

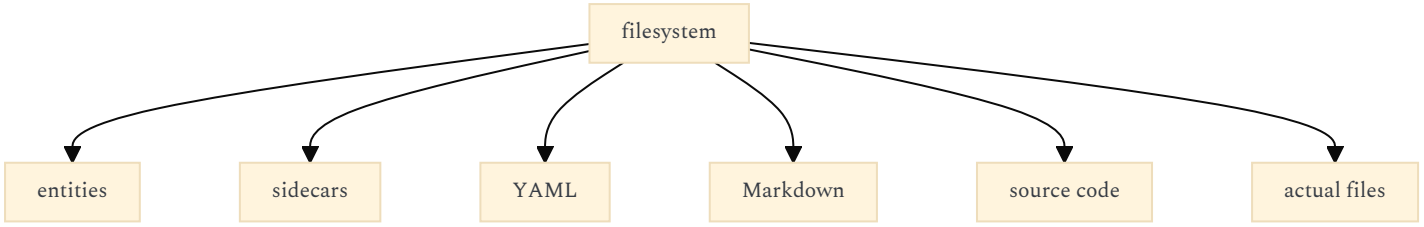
Never:



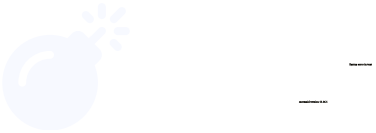
Otherwise incremental rebuilds become dangerous.
Your own `freeze.able`, `assembled.sh`, `compiled.sh` and cache ideas fit better if they remain derived artifacts.

25. I WOULD ALSO REVISE MY VIEW OF SOLR/ARANGO/RAG

The notes make me much more confident that these should remain **secondary projections**.
Your canonical structure could remain:



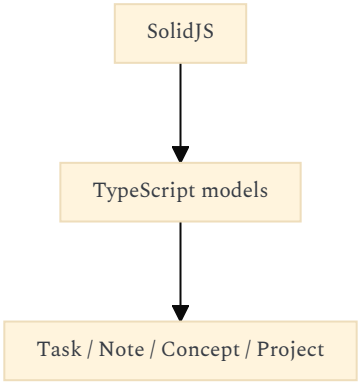
Then:



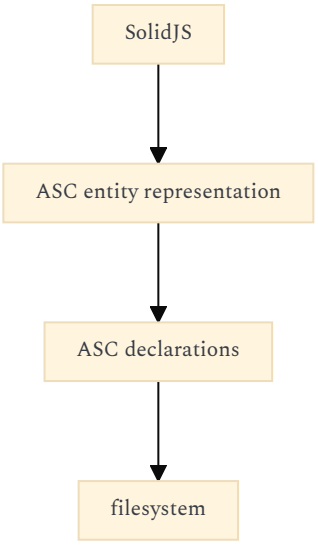
The second brain can use all three without making any of them the ontological center.
This fits your explicit note that mapping ASC relations to complex memory stores is out of scope.

26. ONE VERY IMPORTANT CONSEQUENCE FOR `projet-complexe`

I would now **not create a separate "knowledge model" inside the Tauri application**.
The temptation would be:



Instead:



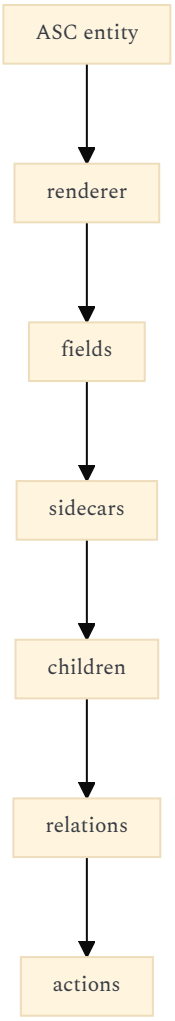
The UI can cache/normalize data for performance, obviously.
But conceptually, the UI should **discover the vocabulary from ASC**.
That allows the UI to display:

- task
- knowledge
- machine
- software
- document
- agent
- change
- file
- project

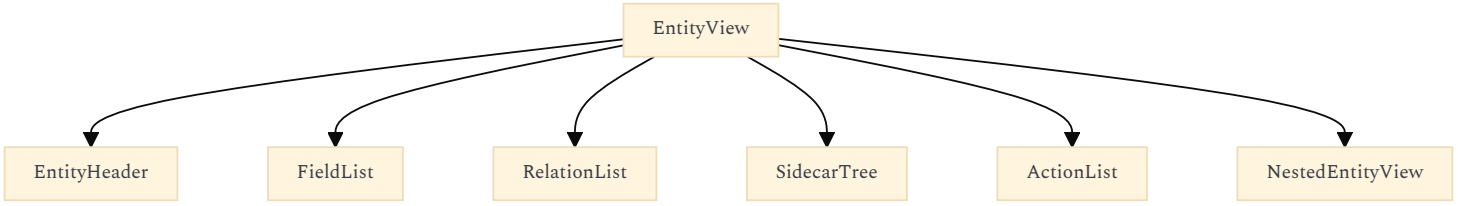
without needing each one hard-coded into the application.

27. THIS IS WHERE KOBALTE/SOLIDJS BECOMES A PARTICULARLY GOOD FIT

Your UI doesn't need to become a giant component framework.
It can be fundamentally:



And then specialized renderers only where necessary.
For example:



with:

- TaskView
- KnowledgeView
- MachineView
- AgentView

being relatively thin semantic projections.
That matches your preference for minimal CSS and typography much better than a component-heavy application model.

28. THE "ATOMIC DESIGN" REFERENCE NOW MAKES SENSE IN A DIFFERENT WAY

Your note about Brad Frost's Atomic Design isn't really about UI components alone.
You're proposing:
atomic.able
for:

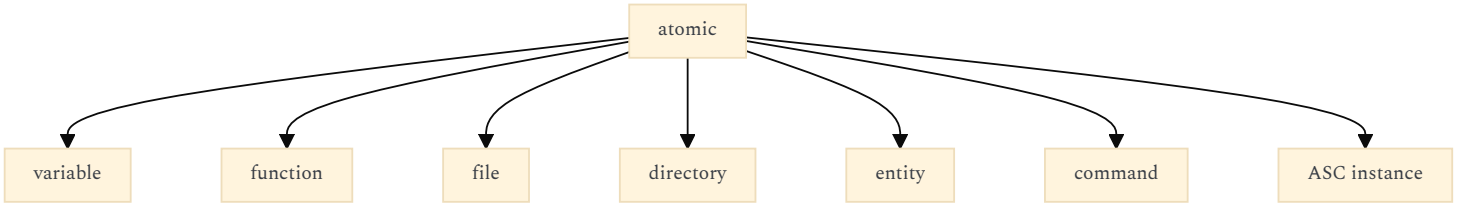
- vars
- functions
- files
- dirs
- ASC instance

That suggests "atomic" in ASC means:

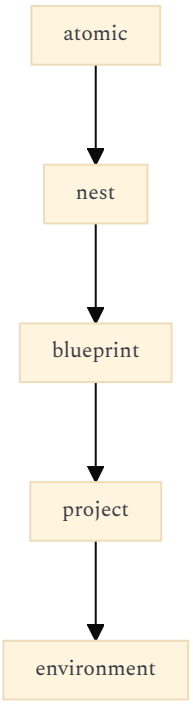
> the smallest structurally addressable/reusable computational unit.

That could eventually become a very powerful common denominator.

For example:



Then higher-level structures are compositions:



This reinforces the fractal/zoomable aspect.

29. YOUR "META" GUIDELINE IS ALSO IMPORTANT FOR THE SECOND BRAIN

This rule:

> never describe the containing thing; describe only what it is about

is exactly the kind of metadata discipline that will matter enormously once you start indexing everything.
For example, avoid:

- type: note
- description: "This is a note about..."

because the filesystem/entity context already establishes `note`.
Instead:
about:
- complex-systems
- agents
The same principle should eventually govern RAG indexing.
Don't inject redundant type descriptions into embeddings if the structural metadata already provides them.

30. REVISED REPOSITORY ARCHITECTURE

With all this taken into account, I'd now draw your ecosystem like this:

This is substantially different from the architecture I proposed previously.
And I think it is **better**.

31. THE KEY ARCHITECTURAL PRINCIPLE I WOULD NOW WRITE INTO THE ASC DOCUMENTATION

I would formulate it approximately like this:

> **ASC does not primarily model software through an external API. It represents computational structures through a filesystem-oriented vocabulary of entities, declarations, sidecars, entry points, hooks and composable DSL expressions.**

> **YAML declares; paths address; DSL formulates; entry points pivot; hooks implement; sidecars concretize; Builder generates; nesting composes; ASC executes.**

That captures much more of what your notes are actually describing.
The resulting correspondence is:

```
YAML       → declaration
path       → identity / address
entity     → virtual thing
sidecar    → concrete representation
field      → instance state
prop       → shared declaration
able       → reusable contract/composition
include    → declaration composition
DSL        → formulation
entrypoint → fixed pivot
hook       → implementation
wrap       → external implementation boundary
nest       → structural composition
builder    → generation
freeze     → derived executable/cache representation
thread     → ASC-managed execution pivot
change     → transformation
```

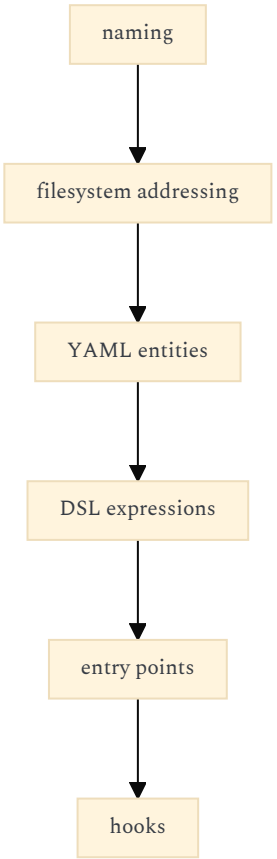
That is, in my view, the real architecture emerging from your notes.

32. AND THIS CHANGES WHAT I WOULD BUILD FIRST

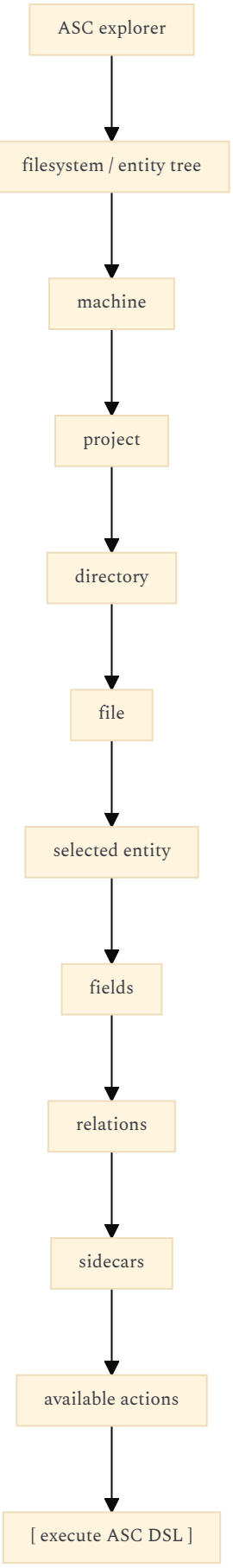
Your current TODO list actually has a very sensible order:

1. naming
2. workflow/git
3. hooks
4. DSL
5. YAML
6. bootstrap
7. core/extensions
8. tests
9. Builder
10. baseline implementations
11. agents

I would **not jump to the Tauri UI before steps 1–5 are reasonably stable**.
The reason is now clearer.
The UI is going to be a major consumer of the semantic invariants established there:



If those move underneath a prematurely built UI, you will end up encoding unstable ASC semantics in TypeScript components.
Instead, the first useful Tauri prototype should probably be almost embarrassingly small:



That would validate the architecture without prematurely building the second brain.

33. FINAL SYNTHESIS

My previous answer had the right **repository separation**, but the wrong center of gravity.
I was thinking:

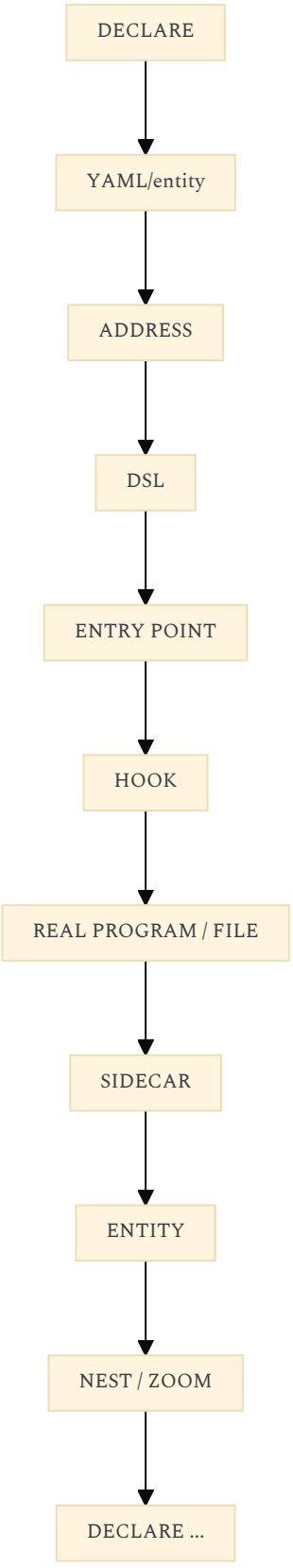
- ASC = execution/control plane

- asc-stack = infrastructure
- projet-complexe = semantic application

After reading your notes, I think the better model is:

- ASC = computational language + filesystem-oriented runtime
- asc-stack = one concrete environment expressed through ASC
- projet-complexe = graphical/semantic projection of that environment, adding its own task/knowledge/agent vocabulary

And the most important conceptual loop is:



That is much closer to what your notes describe than the more conventional "API + services + knowledge database" architecture I previously suggested. The particularly strong idea is that **the same structure can be viewed as code, filesystem, entity, sidecar, DSL, generated artifact, execution, and graphical object depending on the perspective**. That gives `projet-complexe` a genuinely appropriate foundation for becoming the visual second brain and agent-control environment you have in mind, without requiring ASC itself to become second-brain-specific. **Confidence: 0.98.**

CONCLUSION

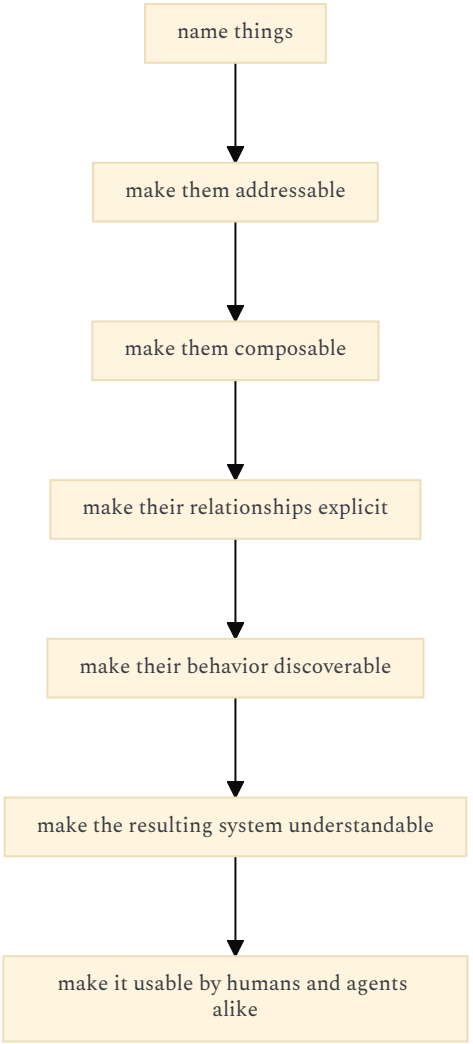
https://chatgpt.com/s/t_6a7d0bf4f704819193956960a3a40085
The beginning of the README already contains the right thesis. I would make the conclusion **less about ASC as a collection of shell utilities** and more about the underlying proposition that emerges from the whole project:

> **Naming is not documentation added after computation. Naming is the structure through which computation becomes composable, inspectable, reproducible, and eventually intelligible to both humans and agents.**

That gives you a natural bridge from the "hard problem of naming things" to the Second Brain / Tauri project without making the README sound like it was secretly designed for that application all along.

PROPOSED CONCLUSION

ASC starts from a deceptively simple problem: **things are difficult to compose when they are difficult to name**.
Files, directories, processes, machines, services, scripts, actions, arguments, environments, dependencies, workflows and capabilities tend to acquire their own vocabularies inside every project. The shell gives us powerful primitives, but very little shared structure for naming, addressing and composing them.
ASC is an attempt to provide such a structure without hiding the shell behind another abstraction layer.
The central idea is therefore not to replace the shell, but to make its vocabulary explicit.
An ASC entity is something that can be named. An entry point is a stable pivot through which something can be addressed or acted upon. A sidecar gives a concrete representation to an otherwise virtual entity. A hook provides a concrete implementation of an abstract operation. A namespace gives names a context. A variant gives an operation alternative implementations. A wrapper or nester composes existing operations rather than reinventing them. YAML declares structure; the filesystem gives that structure a tangible form; the DSL provides a compact way of expressing it; the shell ultimately performs the work.
In that sense, ASC is less a framework than a small language for making the computational environment legible.
This matters increasingly as software stops being written exclusively by humans. Agents can execute commands very well, but execution alone does not give them a coherent model of the environment in which those commands operate. If the things they manipulate have stable names, explicit relationships, discoverable entry points and composable capabilities, then the environment itself becomes something that can be explored, reasoned about and acted upon.
The ambition is consequently recursive:



The same principle applies at larger scales. A project can be represented through its files, scripts, dependencies and processes. A machine can be represented through its hardware, operating system, services and connections. A research corpus can be represented through documents, sources, concepts and transformations. An agent can be represented through its capabilities, tools, threads, tasks, changes and outputs.
These do not need to become different worlds with different APIs. They can become different vocabularies built on the same underlying language.
This is where the current experiments around a Tauri + SolidJS Second Brain eventually lead: not merely to another application for managing notes, tasks or files, but to a visual way of navigating the computational and semantic structures that ASC makes explicit.

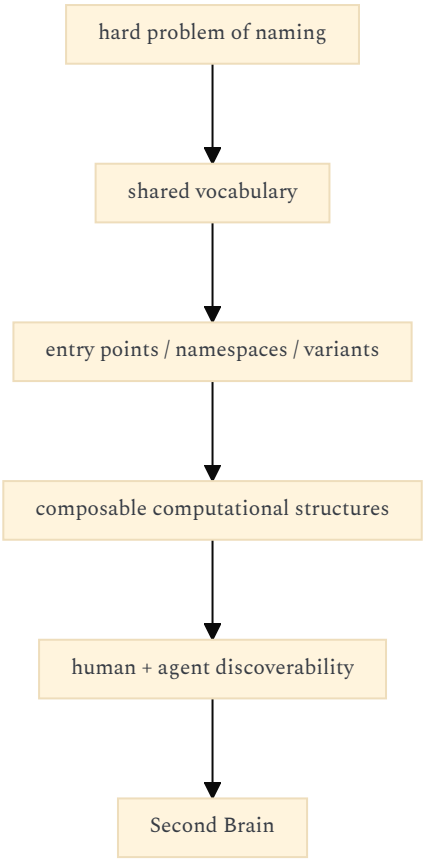
The long-term goal is therefore deliberately broader than automation:

> **to make the things we interact with computable by making them nameable, and to make them understandable by making their names matter.**

Let's make words matter.

Why this fits the README

The important progression is:



It also preserves the particularly strong phrase already present in your README:

> "If you name things right, projects practically write themselves."

The conclusion essentially explains **why that claim can be true**.
The most important sentence, architecturally, is probably:

> "**ASC is less a framework than a small language for making the computational environment legible.**"

That reconciles the apparently disparate parts of ASC: YAML declarations, filesystem conventions, shell scripts, hooks, DSL, Builder, sidecars, threads, workflows, and eventually the Tauri interface.
It also gives the theoretical material in the README a legitimate place without forcing it into the technical architecture: the work on Flow, leverage points, symbolic/neural reasoning, IEML, complex systems, redirection, cognitive institutions, and agent autonomy becomes the **research surrounding the question of what a sufficiently legible computational environment makes possible**, rather than a list of unrelated philosophical interests.
Confidence: 0.96.

Go game analogy

https://chatgpt.com/s/t_6a7d0d5ff17c8191bfc6d17e01484074
No. I actually think the **Go analogy is one of the strongest parts** of the README. I omitted it because I was trying to make the conclusion explain the deeper architectural proposition, but that was a mistake: the analogy gives the reader an immediate mental model for what ASC is trying to do.
The interesting part is that your wording is not merely:

> "ASC is like Go, but for shell commands."

It is more specific:

> "**Like the Go** (**game**), **but with entry points, env vars, scripts** (**wrappers, nesters, 'regular'**), **namespaces and variants** (**hooks**)."

That suggests a much richer analogy.
Go gives you a remarkably small vocabulary:

- stones
- intersections
- groups

- liberties
- territory
- captures
- connections

Yet enormous structures emerge from those primitives.
ASC seems to be pursuing something analogous:

- names
- entities
  entry points
- namespaces
- scripts
- sidecars
- hooks
- arguments
- environment
- nesting
- relations

The important property isn't that either system has few concepts. It's that **the concepts are sufficiently stable and composable that complex configurations can emerge without requiring a new abstraction for every situation**.
That is almost exactly what your README's:

> "If you name things right, projects practically write themselves"

is saying.

I would actually strengthen the analogy

The conclusion could explicitly return to it:

Conclusion

ASC starts from a deceptively simple problem: the hard problem of naming things.
The ambition is to provide a common vocabulary for anything that interacts with the shell somehow: files, directories, processes, machines, services, scripts, actions, environments, dependencies, workflows and, eventually, whatever sits above or below them.
The inspiration is deliberately closer to Go than to another conventional framework:

> **Like the Go game, but for the computational environment.**

Go has a relatively small vocabulary of primitives whose meaning comes from their relationships: stones, intersections, groups, liberties, territory and connections. ASC attempts something analogous for the shell and the systems around it: entities, entry points, namespaces, environments, scripts, wrappers, nesters, variants and hooks.
The objective is not to hide the underlying system behind an elaborate abstraction. It is to make its structure explicit enough that the same primitives can be recombined into increasingly complex structures.
An entity gives something a name. An entry point gives it a stable point of access. A namespace gives that name context. A hook provides a variant implementation. A sidecar gives a virtual entity a concrete representation. A wrapper or nester composes existing behavior. YAML declares structure; the filesystem makes that structure tangible; the DSL provides a compact way of expressing it; the shell ultimately performs the work.
In other words, ASC tries to make the computational environment **playable**.
Not because computation is a game, but because a good vocabulary turns a seemingly unlimited space of possibilities into a space of meaningful moves.
If you name things right, projects practically write themselves.
And this becomes particularly interesting when the player is no longer necessarily human.
An AI agent can execute shell commands, but execution is not the same thing as understanding. A stable vocabulary of entities, relationships, capabilities and entry points gives an agent something closer to a board on which it can orient itself: things have names, positions, relationships, possible moves and consequences.
This is ultimately what the Second Brain project is exploring: a visual environment in which the structures described by ASC can be navigated, inspected, composed and acted upon by humans and agents alike.
The long-term ambition is therefore not simply automation.
It is to make the computational environment sufficiently **nameable, addressable, composable and legible** that increasingly complex projects can emerge from a relatively small set of shared primitives.
Let's make words matter.
I prefer this to my previous version.
The crucial addition is:

> "**ASC tries to make the computational environment playable.**"

That is not saying ASC is literally gamified. It captures the Go analogy at the structural level: **a constrained vocabulary creates an enormous combinatorial space of possible configurations and actions**.
And it connects surprisingly well with the research you have been putting alongside ASC. Your work on Flow, agents, leverage points, complex systems, redirection and cognitive institutions is all circling a related question: **what happens when an agent operates inside an environment whose structure, affordances and possible transformations are made explicit?**
So I would absolutely keep the Go analogy. I would make it **more prominent**, not less.

Confidence: 0.98.